C++ Chapter 04 Notes

Chapter 04 — Notes

4.1 — Introduction to fundamental data types

Bits, bytes, and memory addressing

In lesson 1.3 – Introduction to objects and variables, we talked about the fact that variables are names for a piece of memory that can be used to store information. To recap briefly, computers have random access memory (RAM) that is available for programs to use. When a variable is defined, a piece of that memory is set aside for that variable.

The smallest unit of memory is a **binary digit** (also called a **bit**), which can hold a value of 0 or 1. You can think of a bit as being like a traditional light switch – either the light is off (0), or it is on (1). There is no in-between. If you were to look at a random segment of memory, all you would see is …011010100101010… or some combination thereof.

Memory is organized into sequential units called **memory addresses** (or **addresses** for short). Similar to how a street address can be used to find a given house on a street, the memory address allows us to find and access the contents of memory at a particular location.

Perhaps surprisingly, in modern computer architectures, each bit does not get its own unique memory address. This is because the number of memory addresses is limited, and the need to access data bit-by-bit is rare. Instead, each memory address holds 1 byte of data. A **byte** is a group of bits that are operated on as a unit. The modern standard is that a byte is comprised of 8 sequential bits.

> **Key insight**
>
> In C++, we typically work with "byte-sized" chunks of data.

The following picture shows some sequential memory addresses, along with the corresponding byte of data:

> **As an aside…**
>
> Some older or non-standard machines may have bytes of a different size (from 1 to 48 bits) – however, we generally need not worry about these, as the modern de-facto standard is that a byte is 8 bits. For these tutorials, we'll assume a byte is 8 bits.

Data types

Because all data on a computer is just a sequence of bits, we use a **data type** (often called a **type** for short) to tell the compiler how to interpret the contents of memory in some meaningful way. You have already seen one example of a data type: the integer. When we declare a variable as an integer, we are telling the compiler "the piece of memory that this variable uses is going to be interpreted as an integer value".

When you give an object a value, the compiler and CPU take care of encoding your value into the appropriate sequence of bits for that data type, which are then stored in memory (remember: memory can only store bits). For example, if you assign an integer object the value `65`, that value is converted to the sequence of bits `0100 0001` and stored in the memory assigned to the object.

Conversely, when the object is evaluated to produce a value, that sequence of bits is reconstituted back into the original value. Meaning that `0100 0001` is converted back into the value `65`.

Fortunately, the compiler and CPU do all the hard work here, so you generally don't need to worry about how values get converted into bit sequences and back.

All you need to do is pick a data type for your object that best matches your desired use.

Fundamental data types

The C++ language comes with many predefined data types available for your use. The most basic of these types are called the **fundamental data types** (informally sometimes called **basic types** or **primitive types**).

Here is a list of the fundamental data types, some of which you have already seen:

Types	Category	Meaning	Example
float double long double	Floating Point	a number with a fractional part	3.14159
bool	Integral (Boolean)	true or false	true
char wchar_t char8_t (C++20) char16_t (C++11) char32_t (C++11)	Integral (Character)	a single character of text	'c'
short int int long int long long int (C++11)	Integral (Integer)	positive and negative whole numbers, including 0	64
std::nullptr_t (C++11)	Null Pointer	a null pointer	nullptr
void	Void	no type	n/a

This chapter is dedicated to exploring these fundamental data types in detail (except `std::nullptr_t`, which we'll discuss when we talk about pointers).

Integer vs integral types

In mathematics, an "integer" is a number with no decimal or fractional part, including negative and positive numbers and zero. The term "integral" has several different meanings, but in the context of C++ is used to mean "like an integer".

The C++ standard defines the following terms:

- The **standard integer types** are `short`, `int`, `long`, `long long` (including their signed and unsigned variants).
- The **integral types** are `bool`, the various char types, and the standard integer types.

All integral types are stored in memory as integer values, but only the standard integer types will display as an integer value when output. We'll discuss what `bool` and the char types do when output in their respective lessons.

The C++ standard also explicitly notes that "integer types" is a synonym for "integral types". However, conventionally, "integer types" is more often used as shorthand for the "standard integer types" instead.

Also note that the term "integral types" only includes fundamental types. This means non-fundamental types (such as `enum` and `enum class`) are not integral types, even when they are stored as an integer (and in the case of `enum`, displayed as one too).

Other sets of types

C++ contains three sets of types.

The first two are built-in to the language itself (and do not require the inclusion of a header to use):

- The "fundamental data types" provide the most the basic and essential data types.
- The "compound data types" provide more complex data types and allow for the creation of custom (user-defined) types. We cover these in lesson 12.1 – Introduction to compound data types.

The distinction between the fundamental and compound types isn't all that interesting or relevant, so it's generally fine to consider them as a single set of types.

The third (and largest) set of types is provided by the C++ standard library. Because the standard library is included in all C++ distributions, these types are broadly available and have been standardized for compatibility. Use of the types in the standard library requires the inclusion of the appropriate header and linking in the standard library.

> **Nomenclature**
>
> The term "built-in type" is most often used as a synonym for the fundamental data types. However, Stroustrup and others use the term to mean both the fundamental and compound data types (both of which are built-in to the core language). Since this

> term isn't well-defined, we recommend avoiding it accordingly.

A notable omission from the table of fundamental types above is a data type to handle **strings** (a sequence of characters that is typically used to represent text). This is because in modern C++, strings are part of the standard library. Although we'll be focused on fundamental types in this chapter, because basic string usage is straightforward and useful, we'll introduce strings in the next chapter (in lesson 5.7 – Introduction to std::string).

The _t suffix

Many of the types defined in newer versions of C++ (e.g. `std::nullptr_t`) use a _t suffix. This suffix means "type", and it's a common nomenclature applied to modern types.

If you see something with a _t suffix, it's probably a type. But many types don't have a _t suffix, so this isn't consistently applied.

4.2 — Void

Void is the easiest of the data types to explain. Basically, **void** means "no type"!

Void is our first example of an incomplete type. An **incomplete type** is a type that has been declared but not yet defined. The compiler knows about the existence of such types, but does not have enough information to determine how much memory to allocate for objects of that type. `void` is intentionally incomplete since it represents the lack of a type, and thus cannot be defined.

Incomplete types can not be instantiated:

```
void value; // won't work, variables can't be defined with incomplete type void
```

Void is typically used in several different contexts.

Functions that do not return a value

Most commonly, *void* is used to indicate that a function does not return a value:

```
void writeValue(int x) // void here means no return value
{
    std::cout << "The value of x is: " << x << '\n';
    // no return statement, because this function doesn't return a value
}
```

If you use a return statement to try to return a value in such a function, a compile error will result:

```
void noReturn(int x) // void here means no return value
{
    std::cout << "The value of x is: " << x << '\n';

    return 5; // error
}
```

On Visual Studio 2017, this produced the error:

```
error C2562: 'noReturn': 'void' function returning a value
```

Deprecated: Functions that do not take parameters

In C, void is used as a way to indicate that a function does not take any parameters:

```
int getValue(void) // void here means no parameters
{
    int x{};
    std::cin >> x;

    return x;
}
```

Although this will compile in C++ (for backwards compatibility reasons), this use of keyword *void* is considered deprecated in C++. The following code is equivalent, and preferred in C++:

```
int getValue() // empty function parameters is an implicit void
{
    int x{};
    std::cin >> x;

    return x;
}
```

> **Best practice**
>
> Use an empty parameter list instead of *void* to indicate that a function has no parameters.

Other uses of void

The void keyword has a third (more advanced) use in C++ that we cover in section 19.5 – Void pointers. Since we haven't covered what a pointer is yet, you don't need to worry about this case for now.

Let's move on!

4.3 — Object sizes and the sizeof operator

Object sizes

As you learned in the lesson 4.1 – Introduction to fundamental data types, memory on modern machines is typically organized into byte-sized units, with each byte of memory having a unique address. Up to this point, it has been useful to think of memory as a bunch of cubbyholes or mailboxes where we can put and retrieve information, and variables as names for accessing those cubbyholes or mailboxes.

However, this analogy is not quite correct in one regard – most objects actually take up more than 1 byte of memory. A single object may use 1, 2, 4, 8, or even more consecutive memory addresses. The amount of memory that an object uses is based on its data type.

Because we typically access memory through variable names (and not directly via memory addresses), the compiler is able to hide the details of how many bytes a given object uses from us. When we access some variable `x` in our source code, the compiler knows how many bytes of data need to be retrieved (based on the type of variable `x`), and will output the appropriate machine language code to handle that detail for us.

Even so, there are several reasons it is useful to know how much memory an object uses.

First, the more memory an object uses, the more information it can hold.

A single bit can hold 2 possible values, a 0, or a 1:

bit 0
0
1

2 bits can hold 4 possible values:

bit 0	bit 1
0	0
0	1
1	0
1	1

3 bits can hold 8 possible values:

bit 0	bit 1	bit 2
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

To generalize, an object with *n* bits (where n is an integer) can hold 2n (2 to the power of n, also commonly written 2^n) unique

values. Therefore, with an 8-bit byte, a byte-sized object can hold 28 (256) different values. An object that uses 2 bytes can hold 2^16 (65536) different values!

Thus, the size of the object puts a limit on the amount of unique values it can store – objects that utilize more bytes can store a larger number of unique values. We will explore this further when we talk more about integers.

Second, computers have a finite amount of free memory. Every time we define an object, a small portion of that free memory is used for as long as the object is in existence. Because modern computers have a lot of memory, this impact is usually negligible. However, for programs that need a large amount of objects or data (e.g. a game that is rendering millions of polygons), the difference between using 1 byte and 8 byte objects can be significant.

> **Key insight**
>
> New programmers often focus too much on optimizing their code to use as little memory as possible. In most cases, this makes a negligible difference. Focus on writing maintainable code, and optimize only when and where the benefit will be substantive.

Fundamental data type sizes

The obvious next question is "how much memory do objects of a given data type use?". Perhaps surprisingly, the C++ standard does not define the exact size (in bits) of any of the fundamental types.

Instead, the standard says the following:

- An object must occupy at least 1 byte (so that each object has a distinct memory address).
- A byte must be at least 8 bits.
- The integral types `char`, `short`, `int`, `long`, and `long long` have a minimum size of 8, 16, 16, 32, and 64 bits respectively.
- `char` and `char8_t` are exactly 1 byte (at least 8 bits).

> **Nomenclature**
>
> When we talk about the size of a type, we really mean the size of an instantiated object of that type.

In this tutorial series, we will present a simplified view, by making some reasonable assumptions that are generally true for modern architectures:

- A byte is 8 bits.
- Memory is byte addressable (we can access every byte of memory independently).
- Floating point support is IEEE-754 compliant.
- We are on a 32-bit or 64-bit architecture.

Given the above assumptions, we can reasonably state the following:

Category	Type	Minimum Size	Typical Size
Boolean	bool	1 byte	1 byte
Character	char	1 byte (exactly)	1 byte
	wchar_t	1 byte	2 or 4 bytes
	char8_t	1 byte	1 byte
	char16_t	2 bytes	2 bytes
	char32_t	4 bytes	4 bytes
Integral	short	2 bytes	2 bytes
	int	2 bytes	4 bytes
	long	4 bytes	4 or 8 bytes
	long long	8 bytes	8 bytes
Floating point	float	4 bytes	4 bytes
	double	8 bytes	8 bytes
	long double	8 bytes	8, 12, or 16 bytes
Pointer	std::nullptr_t	4 bytes	4 or 8 bytes

Tip

For maximum portability, you shouldn't assume that objects are larger than the specified minimum size.

Alternatively, if you want to assume that a type has some non-minimum size (e.g. that an int is at least `4` bytes), you can use `static_assert` to have the compiler fail a build if it is compiled on an architecture where this assumption is not true. We cover how to do this in lesson 9.6 – Assert and static_assert.

Related content

You can find more information about what the C++ standard says about the minimum size of various types here.

The sizeof operator

In order to determine the size of data types on a particular machine, C++ provides an operator named `sizeof`. The **sizeof operator** is a unary operator that takes either a type or a variable, and returns the size of an object of that type (in bytes). You can compile and run the following program to find out how large some of your data types are:

```
#include <iomanip> // for std::setw (which sets the width of the subsequent output)
#include <iostream>
#include <climits> // for CHAR_BIT

int main()
{
    std::cout << "A byte is " << CHAR_BIT << " bits\n\n";

    std::cout << std::left; // left justify output

    std::cout << std::setw(16) << "bool:" << sizeof(bool) << " bytes\n";
    std::cout << std::setw(16) << "char:" << sizeof(char) << " bytes\n";
    std::cout << std::setw(16) << "short:" << sizeof(short) << " bytes\n";
    std::cout << std::setw(16) << "int:" << sizeof(int) << " bytes\n";
    std::cout << std::setw(16) << "long:" << sizeof(long) << " bytes\n";
    std::cout << std::setw(16) << "long long:" << sizeof(long long) << " bytes\n";
    std::cout << std::setw(16) << "float:" << sizeof(float) << " bytes\n";
    std::cout << std::setw(16) << "double:" << sizeof(double) << " bytes\n";
    std::cout << std::setw(16) << "long double:" << sizeof(long double) << " bytes\n";

    return 0;
}
```

Here is the output from the author's machine:

```
bool:           1 bytes
char:           1 bytes
short:          2 bytes
int:            4 bytes
long:           4 bytes
long long:      8 bytes
float:          4 bytes
double:         8 bytes
long double:    8 bytes
```

Your results may vary based on compiler, computer architecture, OS, compilation settings (32-bit vs 64-bit), etc…

Trying to use `sizeof` on an incomplete type (such as `void`) will result in a compilation error.

> **For gcc users**
>
> If you have not disabled compiler extensions, gcc allows `sizeof(void)` to return 1 instead of producing a diagnostic (Pointer-Arith). We show how to disable compiler extensions in lesson 0.10 – Configuring your compiler: Compiler extensions.

You can also use the `sizeof` operator on a variable name:

```
#include <iostream>

int main()
{
    int x{};
    std::cout << "x is " << sizeof(x) << " bytes\n";

    return 0;
}
```

```
x is 4 bytes
```

> **For advanced readers**
>
> `sizeof` does not include dynamically allocated memory used by an object. We discuss dynamic memory allocation in a future lesson.

Fundamental data type performance

On modern machines, objects of the fundamental data types are fast, so performance while using or copying these types should generally not be a concern.

> **As an aside…**
>
> You might assume that types that use less memory would be faster than types that use more memory. This is not always true. CPUs are often optimized to process data of a certain size (e.g. 32 bits), and types that match that size may be processed quicker. On such a machine, a 32-bit int could be faster than a 16-bit short or an 8-bit char.

4.4 — Signed integers

An **integer** is an integral type that can represent positive and negative whole numbers, including 0 (e.g. -2, -1, 0, 1, 2). C++ has *4* primary fundamental integer types available for use:

Type	Minimum Size	Note
short int	16 bits	
int	16 bits	Typically 32 bits on modern architectures
long int	32 bits	
long long int	64 bits	

The key difference between the various integer types is that they have varying sizes – the larger integers can hold bigger numbers.

> **A reminder**
>
> C++ only guarantees that integers will have a certain minimum size, not that they will have a specific size. See lesson 4.3 – Object sizes and the sizeof operator for information on how to determine how large each type is on your machine.

> **As an aside…**
>
> Technically, the `bool` and `char` types are considered to be integral types (because these types store their values as integer values). For the purpose of the next few lessons, we'll exclude these types from our discussion.

Signed integers

When writing negative numbers in everyday life, we use a negative sign. For example, *-3* means "negative 3". We'd also typically recognize *+3* as "positive 3" (though common convention dictates that we typically omit plus prefixes).

This attribute of being positive, negative, or zero is called the number's **sign**.

By default, integers in C++ are **signed**, which means the number's sign is stored as part of the value. Therefore, a signed integer can hold both positive and negative numbers (and 0).

In this lesson, we'll focus on signed integers. We'll discuss unsigned integers (which can only hold non-negative numbers) in the next lesson.

Defining signed integers

Here is the preferred way to define the four types of signed integers:

```
short s;      // prefer "short" instead of "short int"
int i;
long l;       // prefer "long" instead of "long int"
long long ll; // prefer "long long" instead of "long long int"
```

Although *short int*, *long int*, or *long long int* will work, we prefer the short names for these types (that do not use the *int* suffix). In addition to being more typing, adding the *int* suffix makes the type harder to distinguish from variables of type *int*. This can lead to mistakes if the short or long modifier is inadvertently missed.

The integer types can also take an optional *signed* keyword, which by convention is typically placed before the type name:

```
signed short ss;
signed int si;
signed long sl;
signed long long sll;
```

However, this keyword should not be used, as it is redundant, since integers are signed by default.

> **Best practice**
>
> Prefer the shorthand types that do not use the `int` suffix or `signed` prefix.

Signed integer ranges

As you learned in the last section, a variable with *n* bits can hold 2^n possible values. But which specific values? We call the set of specific values that a data type can hold its **range**. The range of an integer variable is determined by two factors: its size (in bits), and whether it is signed or not.

For example, an 8-bit signed integer has a range of -128 to 127. This means an 8-bit signed integer can store any integer value between -128 and 127 (inclusive) safely.

Here's a table containing the range of signed integers of different sizes:

Size / Type	Range
8-bit signed	-128 to 127
16-bit signed	-32,768 to 32,767
32-bit signed	-2,147,483,648 to 2,147,483,647
64-bit signed	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

For the math inclined, an n-bit signed variable has a range of -(2^(n-1)) to (2^(n-1))-1.

For the non-math inclined… use the table. :)

> **For advanced readers**
>
> The above ranges assume "two's complement" binary representation. This representation is the de-facto standard for modern architectures (as it is easier to implement in hardware), and is now required by the C++20 standard. We discuss two's complement in lesson O.4 – Converting integers between binary and decimal representation.
>
> In prior standards, sign-magnitude and ones complement representations were permitted for historical reasons. Such representations produce values in the range -(2^(n-1-1)) to +(2^(n-1-1)).

Overflow

What happens if we try to assign the value *140* to an 8-bit signed integer? This number is outside the range that an 8-bit signed integer can hold. The number 140 requires 9 bits to represent (8 magnitude bits and 1 sign bit), but we only have 8 bits (7 magnitude bits and 1 sign bit) available in an 8-bit signed integer.

The C++20 standard makes this blanket statement: "If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined". Colloquially, this is called **overflow**.

Therefore, assigning value 140 to an 8-bit signed integer will result in undefined behavior.

If an arithmetic operation (such as addition or multiplication) attempts to create a value outside the range that can be represented, this is called **integer overflow** (or **arithmetic overflow**). For signed integers, integer overflow will result in undefined behavior.

```
#include <iostream>

int main()
{
    // assume 4 byte integers
    int x { 2'147'483'647 }; // the maximum value of a 4-byte signed integer
    std::cout << x << '\n';

    x = x + 1; // integer overflow, undefined behavior
    std::cout << x << '\n';

    return 0;
}
```

On the author's machine, the above printed:

```
2147483647
-2147483648
```

However, because the second output is the result of undefined behavior, the value output may vary on your machine.

> **For advanced readers**
>
> We cover what happens when unsigned integers overflow in lesson 4.5 – Unsigned integers, and why to avoid them.

In general, overflow results in information being lost, which is almost never desirable. If there is *any* suspicion that an object might need to store a value that falls outside its range, use a type with a bigger range!

Integer division

When dividing two integers, C++ works like you'd expect when the quotient is a whole number:

```
#include <iostream>

int main()
{
    std::cout << 20 / 4 << '\n';
    return 0;
}
```

This produces the expected result:

```
5
```

But let's look at what happens when integer division causes a fractional result:

```
#include <iostream>

int main()
{
    std::cout << 8 / 5 << '\n';
    return 0;
}
```

This produces a possibly unexpected result:

```
1
```

When doing division with two integers (called **integer division**), C++ always produces an integer result. Since integers can't hold fractional values, any fractional portion is simply dropped (not rounded!).

Taking a closer look at the above example, 8 / 5 produces the value 1.6. The fractional part (0.6) is dropped, and the result of 1 remains. Alternatively, we can say 8 / 5 equals 1 remainder 3. The remainder is dropped, leaving 1.

Similarly, -8 / 5 results in the value -1.

> **Warning**
>
> Be careful when using integer division, as you will lose any fractional parts of the quotient. However, if it's what you want, integer division is safe to use, as the results are predictable.

If fractional results are desired, we show a method to do this in lesson 6.2 – Arithmetic operators.

Quiz time

Question 1

What would the range of a 5-bit signed integer be?

> **Show Solution**
>
> A 5-bit number can support 2^5 = 32 unique values. For a signed integer, these are split almost evenly amongst positive and negative numbers, with negative receiving an extra value. So the range would be -16 to 15.
>
> Another way to think about this: 1 of the 5 bits is required to hold the sign. That leaves 4 bits for the magnitude. With 4 bits, we have 16 unique values available. On the negative side, these magnitude bits represent values -1 to -16. On the non-negative side, these magnitude bits represent values 0 to 15. The non-negative side has one less value of range because it includes 0.

Question 2

a. What is the result of 13 / 5?

> **Show Solution**
>
> 13 / 5 = 2 remainder 3. The remainder is dropped, leaving 2.

b. What is the result of -13 / 5?

> **Show Solution**
>
> -13 / 5 = -2 remainder -3. The remainder is dropped, leaving -2.

4.5 — Unsigned integers, and why to avoid them

Unsigned integers

In the previous lesson (4.4 – Signed integers), we covered signed integers, which are a set of types that can hold positive and negative whole numbers, including 0.

C++ also supports unsigned integers. **Unsigned integers** are integers that can only hold non-negative whole numbers.

Defining unsigned integers

To define an unsigned integer, we use the *unsigned* keyword. By convention, this is placed before the type:

```
unsigned short us;
unsigned int ui;
unsigned long ul;
unsigned long long ull;
```

Unsigned integer range

A 1-byte unsigned integer has a range of 0 to 255. Compare this to the 1-byte signed integer range of -128 to 127. Both can store 256 different values, but signed integers use half of their range for negative numbers, whereas unsigned integers can store positive numbers that are twice as large.

Here's a table showing the range for unsigned integers:

Size/Type	Range
8 bit unsigned	0 to 255
16 bit unsigned	0 to 65,535
32 bit unsigned	0 to 4,294,967,295
64 bit unsigned	0 to 18,446,744,073,709,551,615

An n-bit unsigned variable has a range of 0 to (2n)-1.

When no negative numbers are required, unsigned integers are well-suited for networking and systems with little memory, because unsigned integers can store more positive numbers without taking up extra memory.

Remembering the terms signed and unsigned

New programmers sometimes get signed and unsigned mixed up. The following is a simple way to remember the difference: in order to differentiate negative numbers from positive ones, we use a negative sign. If a sign is not provided, we assume a number is positive. Consequently, an integer with a sign (a signed integer) can tell the difference between positive and negative. An integer without a sign (an unsigned integer) assumes all values are positive.

Unsigned integer overflow

What happens if we try to store the number `280` (which requires 9 bits to represent) in a 1-byte (8-bit) unsigned integer? The answer is overflow.

> **Author's note**

> Oddly, the C++ standard explicitly says "a computation involving unsigned operands can never overflow". This is contrary to general programming consensus that integer overflow encompasses both signed and unsigned use cases (cite). Given that most programmers would consider this overflow, we'll call this overflow despite the C++ standard's statements to the contrary.

If an unsigned value is out of range, it is divided by one greater than the largest number of the type, and only the remainder kept.

The number `280` is too big to fit in our 1-byte range of 0 to 255. 1 greater than the largest number of the type is 256. Therefore, we divide 280 by 256, getting 1 remainder 24. The remainder of 24 is what is stored.

Here's another way to think about the same thing. Any number bigger than the largest number representable by the type simply "wraps around" (sometimes called "modulo wrapping"). `255` is in range of a 1-byte integer, so `255` is fine. `256`, however, is outside the range, so it wraps around to the value `0`. `257` wraps around to the value `1`. `280` wraps around to the value `24`.

Let's take a look at this using 2-byte shorts:

```cpp
#include <iostream>

int main()
{
    unsigned short x{ 65535 }; // largest 16-bit unsigned value possible
    std::cout << "x was: " << x << '\n';

    x = 65536; // 65536 is out of our range, so we get modulo wrap-around
    std::cout << "x is now: " << x << '\n';

    x = 65537; // 65537 is out of our range, so we get modulo wrap-around
    std::cout << "x is now: " << x << '\n';

    return 0;
}
```

What do you think the result of this program will be?

(Note: If you try to compile the above program, your compiler should issue warnings about overflow or truncation – you'll need to disable "treat warnings as errors" to run the program)

```
x was: 65535
x is now: 0
x is now: 1
```

It's possible to wrap around the other direction as well. 0 is representable in a 2-byte unsigned integer, so that's fine. -1 is not representable, so it wraps around to the top of the range, producing the value 65535. -2 wraps around to 65534. And so forth.

```
#include <iostream>

int main()
{
    unsigned short x{ 0 }; // smallest 2-byte unsigned value possible
    std::cout << "x was: " << x << '\n';

    x = -1; // -1 is out of our range, so we get modulo wrap-around
    std::cout << "x is now: " << x << '\n';

    x = -2; // -2 is out of our range, so we get modulo wrap-around
    std::cout << "x is now: " << x << '\n';

    return 0;
}
```

```
x was: 0
x is now: 65535
x is now: 65534
```

The above code triggers a warning in some compilers, because the compiler detects that the integer literal is out-of-range for the given type. If you want to compile the code anyway, temporarily disable “Treat warnings as errors”.

> **As an aside…**
>
> Many notable bugs in video game history happened due to wrap around behavior with unsigned integers. In the arcade game Donkey Kong, it’s not possible to go past level 22 due to an overflow bug that leaves the user with not enough bonus time to complete the level.
>
> In the PC game Civilization, Gandhi was known for often being the first one to use nuclear weapons, which seems contrary to his expected passive nature. Players had a theory that Gandhi’s aggression setting was initially set at 1, but if he chose a democratic government, he’d get a -2 aggression modifier (lowering his current aggression value by 2). This would cause his aggression to overflow to 255, making him maximally aggressive! However, more recently Sid Meier (the game’s author) clarified that this wasn’t actually the case.

The controversy over unsigned numbers

Many developers (and some large development houses, such as Google) believe that developers should generally avoid unsigned integers.

This is largely because of two behaviors that can cause problems.

First, with signed values, it takes a little work to accidentally overflow the top or bottom of the range because those values are far from 0. With unsigned numbers, it is much easier to overflow the bottom of the range, because the bottom of the range is 0, which is close to where the majority of our values are.

Consider the subtraction of two unsigned numbers, such as 2 and 3:

```
#include <iostream>

// assume int is 4 bytes
int main()
{
    unsigned int x{ 2 };
    unsigned int y{ 3 };

    std::cout << x - y << '\n'; // prints 4294967295 (incorrect!)

    return 0;
}
```

You and I know that `2 - 3` is `-1`, but `-1` can't be represented as an unsigned integer, so we get overflow and the following result:

```
4294967295
```

Another common unwanted wrap-around happens when an unsigned integer is repeatedly decremented by 1, until it tries to decrement to a negative number. You'll see an example of this when loops are introduced.

Second, and more insidiously, unexpected behavior can result when you mix signed and unsigned integers. In C++, if a mathematical operation (e.g. arithmetic or comparison) has one signed integer and one unsigned integer, the signed integer will usually be converted to an unsigned integer. And the result will thus be unsigned. For example:

```
#include <iostream>

// assume int is 4 bytes
int main()
{
    unsigned int u{ 2 };
    signed int s{ 3 };

    std::cout << u - s << '\n'; // 2 - 3 = 4294967295

    return 0;
}
```

This also produces the result:

```
4294967295
```

In this case, if `u` was signed, the correct result would be produced. But because `u` is unsigned (which is easy to miss), `s` gets converted to unsigned, and the result (`-1`) is treated as an unsigned value. Since `-1` can't be stored in an unsigned value, so we get overflow and an unexpected answer.

Here's another example where things go wrong:

```
#include <iostream>

// assume int is 4 bytes
int main()
{
    signed int s { -1 };
    unsigned int u { 1 };

    if (s < u) // -1 is implicitly converted to 4294967295, and 4294967295 < 1 is false
        std::cout << "-1 is less than 1\n";
    else
        std::cout << "1 is less than -1\n"; // this statement executes

    return 0;
}
```

This prints:

```
1 is less than -1
```

This program is well formed, compiles, and is logically consistent to the eye. But it prints the wrong answer. And while your compiler should warn you about a signed/unsigned mismatch in this case, your compiler will also generate identical warnings for other cases that do not suffer from this problem (e.g. when both numbers are positive), making it hard to detect when there is an actual problem.

> **Related content**
>
> We cover the conversion rules that require both operands of certain binary operations to be the same type in lesson 10.5 – Arithmetic conversions. We cover if-statements in upcoming lesson 4.10 – Introduction to if statements.

Additionally, there are other problematic cases that are challenging to detect. Consider the following:

```
#include <iostream>

// assume int is 4 bytes
void doSomething(unsigned int x)
{
    // Run some code x times

    std::cout << "x is " << x << '\n';
}

int main()
{
    doSomething(-1);

    return 0;
}
```

The author of doSomething() was expecting someone to call this function with only positive numbers. But the caller is passing in *-1* – clearly a mistake, but one made regardless. What happens in this case?

The signed argument of `-1` gets implicitly converted to an unsigned parameter. `-1` isn't in the range of an unsigned number, so it wraps around to 4294967295. Then your program goes ballistic.

Even more problematically, it can be hard to prevent this from happening. Unless you've configured your compiler to be aggressive about producing signed/unsigned conversion warnings (and you should), your compiler probably won't even complain about this.

All of these problems are commonly encountered, produce unexpected behavior, and are hard to find, even using automated tools designed to detect problem cases.

Given the above, the somewhat controversial best practice that we'll advocate for is to avoid unsigned types except in specific circumstances.

> **Best practice**
>
> Favor signed numbers over unsigned numbers for holding quantities (even quantities that should be non-negative) and mathematical operations. Avoid mixing signed and unsigned numbers.

> **Related content**
>
> Additional material in support of the above recommendations (also covers refutation of some common counter-arguments):
>
> 1. Interactive C++ panel (see 9:48-13:08, 41:06-45:26, and 1:02:50-1:03:15)
> 2. Subscripts and sizes should be signed (from Bjarne Stroustrup, the creator of C++)
> 3. Unsigned integers from the libtorrent blog

So when should you use unsigned numbers?

There are still a few cases in C++ where it's okay / necessary to use unsigned numbers.

First, unsigned numbers are preferred when dealing with bit manipulation (covered in chapter O – that's a capital 'o', not a '0'). They are also useful when well-defined wrap-around behavior is required (useful in some algorithms like encryption and random number generation).

Second, use of unsigned numbers is still unavoidable in some cases, mainly those having to do with array indexing. We'll talk more about this in the lessons on arrays and array indexing.

Also note that if you're developing for an embedded system (e.g. an Arduino) or some other processor/memory limited context, use of unsigned numbers is more common and accepted (and in some cases, unavoidable) for performance reasons.

4.6 — Fixed-width integers and size_t

In the previous lessons on integers, we covered that C++ only guarantees that integer variables will have a minimum size – but they could be larger, depending on the target system.

For example, an `int` has a minimum size of 16-bits, but it's typically 32-bits on modern architectures.

If you assume an `int` is 32-bits because that's most likely, then your program will probably misbehave on architectures where `int` is actually 16-bits (since you will probably be storing values that require 32-bits of storage in a variable with only 16-bits of storage, which will cause overflow or undefined behavior).

For example:

```
#include <iostream>

int main()
{
    int x { 32767 };        // x may be 16-bits or 32-bits
    x = x + 1;              // 32768 overflows if int is 16-bits, okay if int is 32-bits
    std::cout << x << '\n'; // what will this print?

    return 0;
}
```

On a machine where `int` is 32-bits, the value `32768` fits within the range of an `int`, and therefore can be stored in `x` without issue. On such a machine, this program will print `32768`. However, on a machine where `int` is 16-bits, the value `32768` does not fit within the range of a 16-bit integer (which has range -32,768 to 32,767). On such a machine, `x = x + 1` will cause overflow, and the value `-32768` will be stored in `x` and then printed.

Instead, if you assume an `int` is only 16-bits to ensure your program will behave on all architectures, then the range of values you can safely store in an `int` is significantly limited. And on systems where `int` is actually 32-bits, you're not making use of half of the memory allocated per `int`.

> **Key insight**
>
> In most cases, we only instantiate a small number of `int` variables at a time, and these are typically destroyed at the end of the function in which they are created. In such cases, wasting 2 bytes of memory per variable isn't a concern (the limited range is a bigger issue). However, in cases where our program allocates millions of `int` variables, wasting 2 bytes of memory per variable can have a significant impact on the program's overall memory usage.

Why isn't the size of the integer types fixed?

The short answer is that this goes back to the early days of C, when computers were slow and performance was of the utmost concern. C opted to intentionally leave the size of an integer open so that the compiler implementers could pick a size for `int` that performs best on the target computer architecture. That way, programmers could just use `int` without having to worry about whether they could be using something more performant.

By modern standards, the lack of consistent ranges for the various integral types sucks (especially in a language designed to be portable).

Fixed-width integers

To address the above issues, C++11 provides an alternate set of integer types that are guaranteed to be the same size on any architecture. Because the size of these integers is fixed, they are called **fixed-width integers**.

The fixed-width integers are defined (in the `<cstdint>` header) as follows:

Name	Fixed Size	Fixed Range	Notes
`std::int8_t`	1 byte signed	-128 to 127	Treated like a signed char on many systems. See note below.
`std::uint8_t`	1 byte unsigned	0 to 255	Treated like an unsigned char on many systems. See note below.
`std::int16_t`	2 byte signed	-32,768 to 32,767	
`std::uint16_t`	2 byte unsigned	0 to 65,535	
`std::int32_t`	4 byte signed	-2,147,483,648 to 2,147,483,647	
`std::uint32_t`	4 byte unsigned	0 to 4,294,967,295	
`std::int64_t`	8 byte signed	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	
`std::uint64_t`	8 byte unsigned	0 to 18,446,744,073,709,551,615	

Here's an example:

```
#include <cstdint> // for fixed-width integers
#include <iostream>

int main()
{
    std::int32_t x { 32767 }; // x is always a 32-bit integer
    x = x + 1;                // so 32768 will always fit
    std::cout << x << '\n';

    return 0;
}
```

> **Best practice**
>
> Use a fixed-width integer type when you need an integral type that has a guaranteed range.

Warning: `std::int8_t` and `std::uint8_t` typically behave like chars

Due to an oversight in the C++ specification, modern compilers typically treat `std::int8_t` and `std::uint8_t` (and the corresponding fast and least fixed-width types, which we'll introduce in a moment) the same as `signed char` and `unsigned char` respectively. Thus on most modern systems, the 8-bit fixed-width integral types will behave like char types.

As a quick teaser:

```
#include <cstdint> // for fixed-width integers
#include <iostream>

int main()
{
    std::int8_t x { 65 };   // initialize 8-bit integral type with value 65
    std::cout << x << '\n'; // You're probably expecting this to print 65

    return 0;
}
```

Although you're probably expecting the above program to print `65`, it most likely won't.

We discuss what this example actually prints (and how to ensure it always prints `65`) in lesson 4.12 – Introduction to type conversion and static_cast, after we cover chars (and how they print) in lesson 4.11 – Chars.

> **Warning**
>
> The 8-bit fixed-width integer types are often treated like chars instead of integer values (and this may vary per system). The 16-bit and wider integral types are not subject to this issue.

> **For advanced readers**
>
> The fixed-width integers actually don't define new types – they are just aliases for existing integral types with the desired size. For each fixed-width type, the implementation (the compiler and standard library) gets to determine which existing type is aliased. As an example, on a platform where `int` is 32-bits, `std::int32_t` will be an alias for `int`. On a system where `int` is 16-bits (and `long` is 32-bits), `std::int32_t` will be an alias for `long` instead.
>
> So what about the 8-bit fixed-width types?
>
> In most cases, `std::int8_t` is an alias for `signed char` because it is the only available 8-bit signed integral type (`bool` and `char` are not considered to be signed integral types). And when this is the case, `std::int8_t` will behave just like a char on that platform.
>
> However, in rare cases, if a platform has an implementation-specific 8-bit signed integral type, the implementation may decide to make `std::int8_t` an alias for that type instead. In that case, `std::int8_t` will behave like that type, which may be more like an int than a char.
>
> `std::uint8_t` behaves similarly.

Other fixed-width downsides

The fixed-width integers have some potential downsides:

First, the fixed-width integers are not guaranteed to be defined on all architectures. They only exist on systems where there are fundamental integral types that match their widths and following a certain binary representation. Your program will fail to compile on any such architecture that does not support a fixed-width integer that your program is using. However, given that modern architectures have standardized around 8/16/32/64-bit variables, this is unlikely to be a problem unless your program needs to be portable to some exotic mainframe or embedded architectures.

Second, if you use a fixed-width integer, it may be slower than a wider type on some architectures. For example, if you need an integer that is guaranteed to be 32-bits, you might decide to use `std::int32_t`, but your CPU might actually be faster at processing 64-bit integers. However, just because your CPU can process a given type faster doesn't mean your program will be faster overall – modern programs are often constrained by memory usage rather than CPU, and the larger memory footprint may slow your program more than the faster CPU processing accelerates it. It's hard to know without actually measuring.

These are just minor quibbles though.

Fast and least integral types [Optional]

To help address the above downsides, C++ also defines two alternative sets of integers that are guaranteed to exist.

The fast types (`std::int_fast#_t` and `std::uint_fast#_t`) provide the fastest signed/unsigned integer type with a width of at least # bits (where # = 8, 16, 32, or 64). For example, `std::int_fast32_t` will give you the fastest signed integer type that's at least 32-bits. By fastest, we mean the integral type that can be processed most quickly by the CPU.

The least types (`std::int_least#_t` and `std::uint_least#_t`) provide the smallest signed/unsigned integer type with a width of at least # bits (where # = 8, 16, 32, or 64). For example, `std::uint_least32_t` will give you the smallest unsigned integer type that's at least 32-bits.

Here's an example from the author's Visual Studio (32-bit console application):

```
#include <cstdint> // for fast and least types
#include <iostream>

int main()
{
    std::cout << "least 8:  " << sizeof(std::int_least8_t)  * 8 << " bits\n";
    std::cout << "least 16: " << sizeof(std::int_least16_t) * 8 << " bits\n";
    std::cout << "least 32: " << sizeof(std::int_least32_t) * 8 << " bits\n";
    std::cout << '\n';
    std::cout << "fast 8:  "  << sizeof(std::int_fast8_t)   * 8 << " bits\n";
    std::cout << "fast 16: "  << sizeof(std::int_fast16_t)  * 8 << " bits\n";
    std::cout << "fast 32: "  << sizeof(std::int_fast32_t)  * 8 << " bits\n";

    return 0;
}
```

This produced the result:

```
least 8:  8 bits
least 16: 16 bits
least 32: 32 bits

fast 8:  8 bits
fast 16: 32 bits
fast 32: 32 bits
```

You can see that `std::int_least16_t` is 16-bits, whereas `std::int_fast16_t` is actually 32-bits. This is because on the author's machine, 32-bit integers are faster to process than 16-bit integers.

As another example, let's assume we're on an architecture that has only 16-bit and 64-bit integral types. `std::int32_t` would not exist, whereas `std::least_int32_t` (and `std::fast_int32_t`) would be 64 bits.

However, these fast and least integers have their own downsides. First, not many programmers actually use them, and a lack of familiarity can lead to errors. Then the fast types can also lead to memory wastage, as their actual size may be significantly larger than indicated by their name.

Most seriously, because the size of the fast/least integers is implementation-defined, your program may exhibit different behaviors on architectures where they resolve to different sizes. For example:

```
#include <cstdint>
#include <iostream>

int main()
{
    std::uint_fast16_t sometype { 0 };
    sometype = sometype - 1; // intentionally overflow to invoke wraparound behavior

    std::cout << sometype << '\n';

    return 0;
}
```

This code will produce different results depending on whether `std::uint_fast16_t` is 16, 32, or 64 bits! This is exactly what we were trying to avoid by using fixed-width integers in the first place!

> **Best practice**
>
> Avoid the fast and least integral types because they may exhibit different behaviors on architectures where they resolve to different sizes.

Best practices for integral types

Given the various pros and cons of the fundamental integral types, the fixed-width integral types, the fast/least integral types, and signed/unsigned challenges, there is little consensus on integral best practices.

Our stance is that it's better to be correct than fast, and better to fail at compile time than runtime. Therefore, if you need an integral type with a guaranteed range, we recommend avoiding the fast/least types in favor of the fixed-width types. If you later discover the need to support an esoteric platform for which a specific fixed-width integral type won't compile, then you can decide how to migrate your program (and thoroughly retest) at that point.

> **Best practice**
>
> - Prefer `int` when the size of the integer doesn't matter (e.g. the number will always fit within the range of a 2-byte signed integer). For example, if you're asking the user to enter their age, or counting from 1 to 10, it doesn't matter whether `int` is 16-bits or 32-bits (the numbers will fit either way). This will cover the vast majority of the cases you're likely to run across.
> - Prefer `std::int#_t` when storing a quantity that needs a guaranteed range.
> - Prefer `std::uint#_t` when doing bit manipulation or well-defined wrap-around behavior is required (e.g. for cryptography or random number generation).
>
> Avoid the following when possible:
>
> - `short` and `long` integers (prefer a fixed-width integer type instead).
> - The fast and least integral types (prefer a fixed-width integer type instead).
> - Unsigned types for holding quantities (prefer a signed integer type instead).
> - The 8-bit fixed-width integer types (prefer a 16-bit fixed-width integer type instead).
> - Any compiler-specific fixed-width integers (for example, Visual Studio defines `__int8`, `__int16`, etc…)

What is std::size_t?

Consider the following code:

```
#include <iostream>

int main()
{
    std::cout << sizeof(int) << '\n';

    return 0;
}
```

On the author's machine, this prints:

```
4
```

Pretty simple, right? We can infer that operator `sizeof` returns an integer value – but what integral type is that return value? An int? A short? The answer is that `sizeof` returns a value of type `std::size_t`. **std::size_t** is an alias for an implementation-defined unsigned integral type. In other words, the compiler decides if `std::size_t` is an unsigned int, an unsigned long, an unsigned long long, etc…

> **Key insight**
>
> `std::size_t` is an alias for an implementation-defined unsigned integral type. It is used within the standard library to represent the byte-size or length of objects.

> **For advanced readers**
>
> `std::size_t` is actually a typedef. We cover typedefs in lesson 10.7 – Typedefs and type aliases.

`std::size_t` is defined in a number of different headers. If you need to use `std::size_t`, `<cstddef>` is the best header to include, as it contains the least number of other defined identifiers.

For example:

```
#include <cstddef>  // for std::size_t
#include <iostream>

int main()
{
    int x { 5 };
    std::size_t s { sizeof(x) }; // sizeof returns a value of type std::size_t, so that should be the type of s
    std::cout << s << '\n';

    return 0;
}
```

> **Best practice**
>
> If you use `std::size_t` explicitly in your code, #include one of the headers that defines `std::size_t` (we recommend `<cstddef>`

4.7 — Introduction to scientific notation

Before we talk about our next subject, we're going to sidebar into the topic of scientific notation.

Scientific notation is a useful shorthand for writing lengthy numbers in a concise manner. And although scientific notation may seem foreign at first, understanding scientific notation will help you understand how floating point numbers work, and more importantly, what their limitations are.

Numbers in scientific notation take the following form: *significand* x 10*exponent*. For example, in the scientific notation `1.2 x 10⁴`, `1.2` is the significand and `4` is the exponent. Since 10^4 evaluates to 10,000, 1.2 x 10^4 evaluates to 12,000.

By convention, numbers in scientific notation are written with one digit before the decimal point, and the rest of the digits afterward.

Consider the mass of the Earth. In decimal notation, we'd write this as `5972200000000000000000000 kg`. That's a really large number (too big to fit even in an 8 byte integer). It's also hard to read (is that 19 or 20 zeros?). Even with separators (5,972,200,000,000,000,000,000,000) the number is still hard to read.

In scientific notation, this would be written as `5.9722 x 10²⁴ kg`, which is much easier to read. Scientific notation has the added benefit of making it easier to compare the magnitude of two extremely large or small numbers simply by comparing the exponent.

Because it can be hard to type or display exponents in C++, we use the letter 'e' (or sometimes 'E') to represent the "times 10 to the power of" part of the equation. For example, `1.2 x 10⁴` would be written as `1.2e4`, and `5.9722 x 10²⁴` would be written as `5.9722e24`.

For numbers smaller than 1, the exponent can be negative. The number `5e-2` is equivalent to `5 * 10⁻²`, which is `5 / 10²`, or `0.05`. The mass of an electron is `9.1093837e-31 kg`.

> **For Linux users**
>
> If you're using arch linux and the superscript of `5 * 10⁻²` is missing the negative sign, you may need to install a font that can display these characters. See this reddit thread for more info.

Significant digits

Let's say you need to know the value of the mathematical constant pi for some equation, but you've forgotten. You ask two people. One tells you the value of pi is `3.14`. The other tells you the value of pi is `3.14159`. Both of these values are "correct", but the latter is far more precise.

Here's the most important thing to understand about scientific notation: The digits in the significand (the part before the 'e') are called the **significant digits** (or **significant figures**). The more significant digits, the more precise a number is.

> **Key insight**
>
> The more digits in the significand, the more precise a number is.

In scientific notation, we'd write `3.14` as `3.14e0`. Since there are 3 numbers in the significand, this number has 3 significant digits.

`3.14159` would be written as `3.14159e0`. Since there are 6 numbers in the significand, this number has 6 significant digits.

How to convert decimal numbers to scientific notation

Use the following procedure:

- Your exponent starts at zero.
- If the number has no explicit decimal point (e.g. `123`), it is implicitly on the right end (e.g. `123.`)
- Slide the decimal point left or right so there is only one non-zero digit to the left of the decimal.

- Each place you slide the decimal point to the left increases the exponent by 1.
- Each place you slide the decimal point to the right decreases the exponent by 1.
- Trim off any leading zeros (on the left end of the significand)
- Trim off any trailing zeros (on the right end of the significand) only if the original number had no decimal point. We're assuming they're not significant. If you have additional information to suggest they are significant, you can keep them.

Here's some examples:

```
Start with: 600.410
Slide decimal point left 2 spaces: 6.00410e2
No leading zeros to trim: 6.00410e2
Don't trim trailing zeros: 6.00410e2 (6 significant digits)
```

```
Start with: 0.0078900
Slide decimal point right 3 spaces: 0007.8900e-3
Trim leading zeros: 7.8900e-3
Don't trim trailing zeros: 7.8900e-3 (5 significant digits)
```

```
Start with: 42030 (no information to suggest the trailing zero is significant)
Slide decimal point left 4 spaces: 4.2030e4
No leading zeros to trim: 4.2030e4
Trim trailing zeros: 4.203e4 (4 significant digits)
```

```
Start with: 42030 (assuming the trailing zero is significant)
Slide decimal point left 4 spaces: 4.2030e4
No leading zeros to trim: 4.2030e4
Keep trailing zeros: 4.2030e4 (5 significant digits)
```

Handling trailing zeros

Consider the case where we ask two lab assistants each to weigh the same apple. One returns and says the apple weighs 87.0 grams. The other returns and says the apple weighs 87.000 grams. Let's assume the weighing is correct. In the former case, the actual weight of the apple could be anywhere between 86.950 and 87.049 grams. Maybe the scale was only precise to the nearest tenth of a gram. Or maybe our assistant rounded a bit. In the latter case, we are confident about the actual weight of the apple to a much higher degree (it actually weighs between 86.99950 and 87.00049 grams, which has much less variability).

When converting to scientific notation, trailing zeros after a decimal point are considered significant, so we keep them:

- 87.0g = 8.70e1
- 87.000g = 8.7000e1

For numbers with no decimal point, trailing zeros are considered to be insignificant by default. Given the number 2100 (with no additional information), we assume the trailing zeroes are not significant, so we drop them:

- 2100 = 2.1e3 (trailing zeros assumed not significant)

However, if we happened to know that this number was measured precisely (or that the actual number was somewhere between 2099.5 and 2100.5), then we should instead treat those zeros as significant:

- 2100 = 2.100e3 (trailing zeros known significant)

> **Tip**

> You may occasionally see a number written with an explicit trailing decimal point. This is an indication that the preceding zeros are significant.
>
> 2100. = 2.100e3 (trailing zeros known significant)

One of the nice things about scientific notation is that it always makes clear how many digits are significant.

From a technical standpoint, the numbers 87.0 and 87.000 have the same value (and the same type). When C++ stores either of these numbers in memory, it will store just the value 87. And once stored, there is no way to determine from the stored value which of the two numbers was originally input.

Now that we've covered scientific notation, we're ready to cover floating point numbers.

4.8 — Floating point numbers

Integers are great for counting whole numbers, but sometimes we need to store *very* large (positive or negative) numbers, or numbers with a fractional component. A **floating point** type variable is a variable that can hold a number with a fractional component, such as 4320.0, -3.33, or 0.01226. The *floating* part of the name *floating point* refers to the fact that the decimal point can "float" – that is, it can support a variable number of digits before and after the decimal point. Floating point data types are always signed (can hold positive and negative values).

> **Tip**
>
> When writing floating point numbers in your code, the decimal separator must be a decimal point. If you're from a country that uses a decimal comma, you'll need to get used to using a decimal point instead.

C++ floating point types

C++ has three fundamental floating point data types: a single-precision `float`, a double-precision `double`, and an extended-precision `long double`. As with integers, C++ does not define the actual size of these types.

Category	C++ Type	Typical Size
floating point	float	4 bytes
	double	8 bytes
	long double	8, 12, or 16 bytes

On modern architectures, floating-point types are conventionally implemented using one of the floating-point formats defined in the IEEE 754 standard (see https://en.wikipedia.org/wiki/IEEE_754). As a result, `float` is almost always 4 bytes, and `double` is almost always 8 bytes.

On the other hand, `long double` is a strange type. On different platforms, its size can vary between 8 and 16 bytes, and it may or may not use an IEEE 754 compliant format. We recommend avoiding `long double`.

> **Tip**
>
> This tutorial series assumes your compiler is using an IEEE 754 compatible format for `float` and `double`.
>
> You can see if your floating point types are IEEE 754 compatible with the following code:

```
#include <iostream>
#include <limits>

int main()
{
    std::cout << std::boolalpha; // print bool as true or false rather than 1 or 0
    std::cout << "float: " << std::numeric_limits<float>::is_iec559 << '\n';
    std::cout << "double: " << std::numeric_limits<double>::is_iec559 << '\n';
    std::cout << "long double: " << std::numeric_limits<long double>::is_iec559 << '\n';
}
```

> **For advanced readers**
>
> `float` is almost always implemented using the 4-byte IEEE 754 single-precision format. `double` is almost always implemented using the 8-byte IEEE 754 double-precision format.

> However, the format used to implement `long double` varies by platform. Common choices include:
>
> - 8-byte IEEE 754 double-precision format (same as `double`).
> - 80-bit (often padded to 12 or 16 bytes) x87 extended-precision format (compatible with IEEE 754).
> - 16-byte IEEE 754 quadruple-precision format.
> - 16-byte double-double format (not compatible with IEEE 754).

Floating point variables and literals

Here are some definitions of floating point variables:

```
float f;
double d;
long double ld;
```

When using floating point literals, always include at least one decimal place (even if the decimal is 0). This helps the compiler understand that the number is a floating point number and not an integer.

```
int a { 5 };      // 5 means integer
double b { 5.0 }; // 5.0 is a floating point literal (no suffix means double type by default)
float c { 5.0f }; // 5.0 is a floating point literal, f suffix means float type

int d { 0 };      // 0 is an integer
double e { 0.0 }; // 0.0 is a double
```

Note that by default, floating point literals default to type double. An `f` suffix is used to denote a literal of type float.

> **Best practice**
>
> Always make sure the type of your literals match the type of the variables they're being assigned to or used to initialize. Otherwise an unnecessary conversion will result, possibly with a loss of precision.

Printing floating point numbers

Now consider this simple program:

```
#include <iostream>

int main()
{
    std::cout << 5.0 << '\n';
    std::cout << 6.7f << '\n';
    std::cout << 9876543.21 << '\n';

    return 0;
}
```

The results of this seemingly simple program may surprise you:

```
5
6.7
9.87654e+06
```

In the first case, `std::cout` printed `5`, even though we typed in `5.0`. By default, `std::cout` will not print the fractional part of a number if the fractional part is 0.

In the second case, the number prints as we expect.

In the third case, it printed the number in scientific notation (if you need a refresher on scientific notation, see lesson 4.7 – Introduction to scientific notation).

Floating point range

Format	Range	Precision
IEEE 754 single-precision (4 bytes)	$\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ and 0.0	6-9 significant digits, typically 7
IEEE 754 double-precision (8 bytes)	$\pm 2.23 \times 10^{-308}$ to $\pm 1.80 \times 10^{308}$ and 0.0	15-18 significant digits, typically 16
x87 extended-precision (80 bits)	$\pm 3.36 \times 10^{-4932}$ to $\pm 1.18 \times 10^{4932}$ and 0.0	18-21 significant digits
IEEE 754 quadruple-precision (16 bytes)	$\pm 3.36 \times 10^{-4932}$ to $\pm 1.18 \times 10^{4932}$ and 0.0	33-36 significant digits

For advanced readers

The 80-bit x87 extended-precision floating point type is a bit of a historical anomaly. On modern processors, objects of this type are typically padded to 12 or 16 bytes (which is a more natural size for processors to handle). This means those objects have 80-bits of floating point data, and the remaining memory is filler.

It may seem a little odd that the 80-bit floating point type has the same range as the 16-byte floating point type. This is because they have the same number of bits dedicated to the exponent – however, the 16-byte number can store more significant digits.

Floating point precision

Consider the fraction 1/3. The decimal representation of this number is 0.33333333333333… with 3's going out to infinity. If you were writing this number on a piece of paper, your arm would get tired at some point, and you'd eventually stop writing. And the number you were left with would be close to 0.3333333333…. (with 3's going out to infinity) but not exactly.

On a computer, an infinite precision number would require infinite memory to store, and we typically only have 4 or 8 bytes per value. This limited memory means floating point numbers can only store a certain number of significant digits – any additional significant digits are either lost or represented imprecisely. The number that is actually stored may be close to the desired number, but not exact. We'll show an example of this in the next section.

The **precision** of a floating point type defines how many significant digits it can represent without information loss.

The number of digits of precision a floating point type has depends on both the size (floats have less precision than doubles) and the particular value being stored (some values can be represented more precisely than others).

For example, a float has 6 to 9 digits of precision. This means that a float can exactly represent any number with up to 6 significant digits. A number with 7 to 9 significant digits may or may not be represented exactly depending on the specific value. And a number with more than 9 digits of precision will definitely not be represented exactly.

Double values have between 15 and 18 digits of precision, with most double values having at least 16 significant digits. Long double has a minimum precision of 15, 18, or 33 significant digits depending on how many bytes it occupies.

Key insight

> A floating point type can only precisely represent a certain number of significant digits. Using a value with more significant digits than the minimum may result in the value being stored inexactly.

Outputting floating point values

When outputting floating point numbers, `std::cout` has a default precision of 6 – that is, it assumes all floating point variables are only significant to 6 digits (the minimum precision of a float), and hence it will truncate anything after that.

The following program shows `std::cout` truncating to 6 digits:

```
#include <iostream>

int main()
{
    std::cout << 9.87654321f << '\n';
    std::cout << 987.654321f << '\n';
    std::cout << 987654.321f << '\n';
    std::cout << 9876543.21f << '\n';
    std::cout << 0.0000987654321f << '\n';

    return 0;
}
```

This program outputs:

```
9.87654
987.654
987654
9.87654e+006
9.87654e-005
```

Note that each of these only have 6 significant digits.

Also note that std::cout will switch to outputting numbers in scientific notation in some cases. Depending on the compiler, the exponent will typically be padded to a minimum number of digits. Fear not, 9.87654e+006 is the same as 9.87654e6, just with some padding 0's. The minimum number of exponent digits displayed is compiler-specific (Visual Studio uses 3, some others use 2 as per the C99 standard).

We can override the default precision that std::cout shows by using an `output manipulator` function named `std::setprecision()`. **Output manipulators** alter how data is output, and are defined in the *iomanip* header.

```
#include <iomanip> // for output manipulator std::setprecision()
#include <iostream>

int main()
{
    std::cout << std::setprecision(17); // show 17 digits of precision
    std::cout << 3.3333333333333333333333333333333333333f <<'\n'; // f suffix means float
    std::cout << 3.3333333333333333333333333333333333333 << '\n'; // no suffix means double

    return 0;
}
```

Outputs:

```
3.3333332538604736
3.3333333333333335
```

Because we set the precision to 17 digits using `std::setprecision()`, each of the above numbers is printed with 17 digits. But, as you can see, the numbers certainly aren't precise to 17 digits! And because floats are less precise than doubles, the float has more error.

> **Tip**
>
> Output manipulators (and input manipulators) are sticky – meaning if you set them, they will remain set.
>
> The one exception is `std::setw`. Some IO operations reset `std::setw`, so `std::setw` should be used every time it is needed.

Precision issues don't just impact fractional numbers, they impact any number with too many significant digits. Let's consider a big number:

```
#include <iomanip> // for std::setprecision()
#include <iostream>

int main()
{
    float f { 123456789.0f }; // f has 10 significant digits
    std::cout << std::setprecision(9); // to show 9 digits in f
    std::cout << f << '\n';

    return 0;
}
```

Output:

```
123456792
```

123456792 is greater than 123456789. The value 123456789.0 has 10 significant digits, but float values typically have 7 digits of precision (and the result of 123456792 is precise only to 7 significant digits). We lost some precision! When precision is lost because a number can't be stored precisely, this is called a **rounding error**.

Consequently, one has to be careful when using floating point numbers that require more precision than the variables can hold.

> **Best practice**
>
> Favor double over float unless space is at a premium, as the lack of precision in a float will often lead to inaccuracies.

Rounding errors make floating point comparisons tricky

Floating point numbers are tricky to work with due to non-obvious differences between binary (how data is stored) and decimal (how we think) numbers. Consider the fraction 1/10. In decimal, this is easily represented as 0.1, and we are used to thinking of 0.1 as an easily representable number with 1 significant digit. However, in binary, decimal value 0.1 is represented by the infinite sequence: 0.00011001100110011… Because of this, when we assign 0.1 to a floating point number, we'll run into precision problems.

You can see the effects of this in the following program:

```
#include <iomanip> // for std::setprecision()
#include <iostream>

int main()
{
    double d{0.1};
    std::cout << d << '\n'; // use default cout precision of 6
    std::cout << std::setprecision(17);
    std::cout << d << '\n';

    return 0;
}
```

This outputs:

```
0.1
0.10000000000000001
```

On the top line, `std::cout` prints 0.1, as we expect.

On the bottom line, where we have `std::cout` show us 17 digits of precision, we see that `d` is actually *not quite* 0.1! This is because the double had to truncate the approximation due to its limited memory. The result is a number that is precise to 16 significant digits (which type double guarantees), but the number is not *exactly* 0.1. Rounding errors may make a number either slightly smaller or slightly larger, depending on where the truncation happens.

Rounding errors can have unexpected consequences:

```
#include <iomanip> // for std::setprecision()
#include <iostream>

int main()
{
    std::cout << std::setprecision(17);

    double d1{ 1.0 };
    std::cout << d1 << '\n';

    double d2{ 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 }; // should equal 1.0
    std:::cout << d2 << '\n';

    return 0;
}
```

```
1
0.99999999999999989
```

Although we might expect that `d1` and `d2` should be equal, we see that they are not. If we were to compare `d1` and `d2` in a program, the program would probably not perform as expected. Because floating point numbers tend to be inexact, comparing floating point numbers is generally problematic – we discuss the subject more (and solutions) in lesson 6.7 – Relational operators and floating point comparisons.

One last note on rounding errors: mathematical operations (such as addition and multiplication) tend to make rounding errors grow.

So even though 0.1 has a rounding error in the 17th significant digit, when we add 0.1 ten times, the rounding error has crept into the 16th significant digit. Continued operations would cause this error to become increasingly significant.

> **Key insight**
>
> Rounding errors occur when a number can't be stored precisely. This can happen even with simple numbers, like 0.1. Therefore, rounding errors can, and do, happen all the time. Rounding errors aren't the exception – they're the norm. Never assume your floating point numbers are exact.
>
> A corollary of this

4.9 — Boolean values

In real-life, it's common to ask or be asked questions that can be answered with "yes" or "no". "Is an apple a fruit?" Yes. "Do you like asparagus?" No.

Now consider a similar statement that can be answered with a "true" or "false": "Apples are a fruit". It's clearly true. Or how about, "I like asparagus". Absolutely false (yuck!).

These kinds of sentences that have only two possible outcomes: yes/true, or no/false are so common, that many programming languages include a special type for dealing with them. That type is called a **Boolean** type (note: Boolean is properly capitalized in the English language because it's named after its inventor, George Boole).

Boolean variables

Boolean variables are variables that can have only two possible values: `true`, and `false`.

To declare a Boolean variable, we use the keyword `bool`.

```
bool b;
```

To initialize or assign a `true` or `false` value to a Boolean variable, we use the keywords `true` and `false`.

```
bool b1 { true };
bool b2 { false };
b1 = false;
bool b3 {}; // default initialize to false
```

Just as the unary minus operator (`-`) can be used to make an integer negative, the logical NOT operator (`!`) can be used to flip a Boolean value from `true` to `false`, or `false` to `true`:

```
bool b1 { !true }; // b1 will be initialized with the value false
bool b2 { !false }; // b2 will be initialized with the value true
```

Boolean values are not actually stored in Boolean variables as the words "true" or "false". Instead, they are stored as integral values: `true` is stored as integer `1`, and `false` is stored as integer `0`. Similarly, when Boolean values are evaluated, they don't actually evaluate to "true" or "false". They evaluate to the integers `0` (false) or `1` (true). Because Booleans store integral values, they are considered to be an integral type.

Printing Boolean values

When we print Boolean values, `std::cout` prints `0` for `false`, and `1` for `true`:

```
#include <iostream>

int main()
{
    std::cout << true << '\n'; // true evaluates to 1
    std::cout << !true << '\n'; // !true evaluates to 0

    bool b {false};
    std::cout << b << '\n'; // b is false, which evaluates to 0
    std::cout << !b << '\n'; // !b is true, which evaluates to 1
    return 0;
}
```

Outputs:

```
1
0
0
1
```

Use `std::boolalpha` to print `true` or `false`

If you want `std::cout` to print `true` or `false` instead of `0` or `1`, you can output `std::boolalpha`. This doesn't output anything, but manipulates the way `std::cout` outputs `bool` values.

Here's an example:

```
#include <iostream>

int main()
{
    std::cout << true << '\n';
    std::cout << false << '\n';

    std::cout << std::boolalpha; // print bools as true or false

    std::cout << true << '\n';
    std::cout << false << '\n';
    return 0;
}
```

This prints:

```
1
0
true
false
```

You can use `std::noboolalpha` to turn it back off.

Integer to Boolean conversion

When using uniform initialization, you can initialize a variable using integer literals `0` (for `false`) and `1` (for `true`) (but you really should be using `false` and `true` instead). Other integer literals cause compilation errors:

```
#include <iostream>

int main()
{
    bool bFalse { 0 }; // okay: initialized to false
    bool bTrue  { 1 }; // okay: initialized to true
    bool bNo    { 2 }; // error: narrowing conversions disallowed

    std::cout << bFalse << bTrue << bNo << '\n';

    return 0;
}
```

However, in any context where an integer can be converted to a Boolean, the integer `0` is converted to `false`, and any other integer is converted to `true`.

```
#include <iostream>

int main()
{
    std::cout << std::boolalpha; // print bools as true or false

    bool b1 = 4 ; // copy initialization allows implicit conversion from int to bool
    std::cout << b1 << '\n';

    bool b2 = 0 ; // copy initialization allows implicit conversion from int to bool
    std::cout << b2 << '\n';

    return 0;
}
```

This prints:

```
true
false
```

Note: `bool b1 = 4;` may generate a warning. If so you'll have to disable treating warnings as errors to compile the example.

Inputting Boolean values

Inputting Boolean values using `std::cin` sometimes trips new programmers up.

Consider the following program:

```
#include <iostream>

int main()
{
    bool b{}; // default initialize to false
    std::cout << "Enter a boolean value: ";
    std::cin >> b;
    std::cout << "You entered: " << b << '\n';

    return 0;
}
```

```
Enter a Boolean value: true
You entered: 0
```

Wait, what?

By default, `std::cin` only accepts numeric input for Boolean variables: `0` is `false`, and `1` is `true`. Any other numeric value will be interpreted as `true`, and will cause `std::cin` to enter failure mode. Any non-numeric value will be interpreted as `false` and will cause `std::cin` to enter failure mode.

> **Related content**
>
> We discuss failure mode (and how to get out of it) in lesson 9.5 – std::cin and handling invalid input.

In this case, because we entered `true`, `std::cin` silently failed and set `b` to `false`. Consequently, when `std::cout` prints a value for `b`, it prints `0`.

To allow `std::cin` to accept the words `false` and `true` as inputs, you must first input to `std::boolalpha`:

```
#include <iostream>

int main()
{
    bool b{};
    std::cout << "Enter a boolean value: ";

    // Allow the user to input 'true' or 'false' for boolean values
    // This is case-sensitive, so True or TRUE will not work
    std::cin >> std::boolalpha;
    std::cin >> b;

    // Let's also output bool values as `true` or `false`
    std::cout << std::boolalpha;
    std::cout << "You entered: " << b << '\n';

    return 0;
}
```

However, when `std::boolalpha` is enabled for input, numeric values will no longer be accepted (they evaluate to `false` and cause std::cin to enter failure mode).

> **Warning**
>
> Enabling `std::boolalpha` for input will only allow lower-cased `false` or `true` to be accepted. Variations with capital letters will not be accepted. `0` and `1` will also no longer be accepted.

Note that we use `std::cin >> std::boolalpha;` to input bool values as `true` or `false`, and `std::cout << std::boolalpha;` to output bool values as `true` or `false`. These are independent controls that can be turned on (using `std::boolalpha`) or off (using `std::noboolalpha`) separately.

Boolean return values

Boolean values are often used as the return values for functions that check whether something is true or not. Such functions are typically named starting with the word *is* (e.g. isEqual) or *has* (e.g. hasCommonDivisor).

Consider the following example, which is quite similar to the above:

```
#include <iostream>

// returns true if x and y are equal, false otherwise
bool isEqual(int x, int y)
{
    return x == y; // operator== returns true if x equals y, and false otherwise
}

int main()
{
    std::cout << "Enter an integer: ";
    int x{};
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y{};
    std::cin >> y;

    std::cout << std::boolalpha; // print bools as true or false

    std::cout << x << " and " << y << " are equal? ";
    std::cout << isEqual(x, y) << '\n'; // will return true or false

    return 0;
}
```

Here's output from two runs of this program:

```
Enter an integer: 5
Enter another integer: 5
5 and 5 are equal? true
```

```
Enter an integer: 6
Enter another integer: 4
6 and 4 are equal? false
```

How does this work? First we read in integer values for `x` and `y`. Next, the expression `isEqual(x, y)` is evaluated. In the first run, this results in a function call to `isEqual(5, 5)`. Inside that function, `5 == 5` is evaluated, producing the value `true`. The value `true` is returned back to the caller to be printed by `std::cout`. In the second run, the call to `isEqual(6, 4)` returns the value `false`.

Boolean values take a little bit of getting used to, but once you get your mind wrapped around them, they're quite refreshing in their simplicity! Boolean values are also a huge part of the language – you'll end up using them more than all the other fundamental types put together!

We'll continue our exploration of Boolean values in the next lesson.

4.10 — Introduction to if statements

Consider a case where you're going to go to the market, and your roommate tells you, "if they have strawberries on sale, buy some". This is a conditional statement, meaning that you'll execute some action ("buy some") only if the condition ("they have strawberries on sale") is true.

Such conditions are common in programming, as they allow us to implement conditional behavior into our programs. The simplest kind of conditional statement in C++ is called an *if statement*. An **if statement** allows us to execute one (or more) lines of code only if some condition is true.

The simplest *if statement* takes the following form:

```
if (condition) true_statement;
```

For readability, this is more often written as following:

```
if (condition)
    true_statement;
```

A **condition** (also called a **conditional expression**) is an expression that evaluates to a Boolean value.

If the *condition* of an *if statement* evaluates to Boolean value *true*, then *true_statement* is executed. If the *condition* instead evaluates to Boolean value *false*, then *true_statement* is skipped.

A sample program using an if statement

Given the following program:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    if (x == 0)
        std::cout << "The value is zero\n";

    return 0;
}
```

Here's output from one run of this program:

```
Enter an integer: 0
The value is zero
```

Let's examine how this works in more detail.

First, the user enters an integer. Then the condition *x == 0* is evaluated. The *equality operator* (==) is used to test whether two values are equal. Operator== returns *true* if the operands are equal, and *false* if they are not. Since *x* has value 0, and *0 == 0* is true, this expression evaluates to *true*.

Because the condition has evaluated to *true*, the subsequent statement executes, printing *The value is zero*.

Here's another run of this program:

```
Enter an integer: 5
```

In this case, *x == 0* evaluates to *false*. The subsequent statement is skipped, the program ends, and nothing else is printed.

> **Warning**
>
> *If statements* only conditionally execute a single statement. We talk about how to conditionally execute multiple statements in lesson 8.2 – If statements and blocks.

If-else

Given the above example, what if we wanted to tell the user that the number they entered was non-zero?

We could write something like this:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    if (x == 0)
        std::cout << "The value is zero\n";
    if (x != 0)
        std::cout << "The value is non-zero\n";

    return 0;
}
```

Or this:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    bool zero { (x == 0) };
    if (zero)
        std::cout << "The value is zero\n";
    if (!zero)
        std::cout << "The value is non-zero\n";

    return 0;
}
```

Both of these programs are more complex than they need to be. Instead, we can use an alternative form of the *if statement* called *if-else*. *If-else* takes the following form:

```
if (condition)
    true_statement;
else
    false_statement;
```

If the *condition* evaluates to Boolean true, *true_statement* executes. Otherwise *false_statement* executes.

Let's amend our previous program to use an *if-else*.

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    if (x == 0)
        std::cout << "The value is zero\n";
    else
        std::cout << "The value is non-zero\n";

    return 0;
}
```

Now our program will produce the following output:

```
Enter an integer: 0
The value is zero
```

```
Enter an integer: 5
The value is non-zero
```

Chaining if statements

Sometimes we want to check if several things are true or false in sequence. We can do so by chaining an *if-statement* (or *if-else*) to a prior *if-else*, like so:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    if (x > 0)
        std::cout << "The value is positive\n";
    else if (x < 0)
        std::cout << "The value is negative\n";
    else
        std::cout << "The value is zero\n";

    return 0;
}
```

The *less than operator* (<) is used to test whether one value is less than another. Similarly, the *greater than operator* (>) is used to test whether one value is greater than another. These operators both return Boolean values.

Here's output from a few runs of this program:

```
Enter an integer: 4
The value is positive
```

```
Enter an integer: -3
The value is negative
```

```
Enter an integer: 0
The value is zero
```

Note that you can chain *if statements* as many times as you have conditions you want to evaluate. We'll see an example in the quiz where this is useful.

Boolean return values and if statements

In the previous lesson (4.9 – Boolean values), we wrote this program using a function that returns a Boolean value:

```cpp
#include <iostream>

// returns true if x and y are equal, false otherwise
bool isEqual(int x, int y)
{
    return x == y; // operator== returns true if x equals y, and false otherwise
}

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y {};
    std::cin >> y;

    std::cout << std::boolalpha; // print bools as true or false

    std::cout << x << " and " << y << " are equal? ";
    std::cout << isEqual(x, y); // will return true or false

    std::cout << '\n';

    return 0;
}
```

Let's improve this program using an *if statement*:

```
#include <iostream>

// returns true if x and y are equal, false otherwise
bool isEqual(int x, int y)
{
    return x == y; // operator== returns true if x equals y, and false otherwise
}

int main()
{
    std::cout << "Enter an integer: ";
    int x {};
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y {};
    std::cin >> y;

    if (isEqual(x, y))
        std::cout << x << " and " << y << " are equal\n";
    else
        std::cout << x << " and " << y << " are not equal\n";

    return 0;
}
```

Two runs of this program:

```
Enter an integer: 5
Enter another integer: 5
5 and 5 are equal
```

```
Enter an integer: 6
Enter another integer: 4
6 and 4 are not equal
```

In this case, our conditional expression is simply a function call to function *isEqual*, which returns a Boolean value.

Non-Boolean conditionals

In all of the examples above, our conditionals have been either Boolean values (`true` or `false`), Boolean variables, or functions that return a Boolean value. What happens if your conditional is an expression that does not evaluate to a Boolean value?

In such a case, the result of the conditional expression is converted to a Boolean value: non-zero values get converted to Boolean `true`, and zero-values get converted to Boolean `false`.

Therefore, if we do something like this:

```
#include <iostream>

int main()
{
    int x { 4 };
    if (x) // nonsensical, but for the sake of example...
        std::cout << "hi\n";
    else
        std::cout << "bye\n";

    return 0;
}
```

This will print `hi`, since `x` has value `4`, and `4` is a non-zero value that gets converted to Boolean `true`, causing the statement after the condition to execute.

> **Key insight**
>
> `if (x)` means "if x is non-zero/non-empty".

If-statements and early returns

A return-statement that is not the last statement in a function is called an **early return**. Such a statement will cause the function to return to the caller when the return statement is executed (before the function would otherwise return to the caller, hence, "early").

An unconditional early return is not useful:

```
void print()
{
    std::cout << "A" << '\n';

    return; // the function will always return to the caller here

    std::cout << "B" << '\n'; // this will never be printed
}
```

Since `std::cout << "B" << '\n';` will never be executed, we might as well remove it, and then our `return` statement is no longer early.

However, when combined with if-statements, early returns provide a way to conditionalize the return value of our function.

```
#include <iostream>

// returns the absolute value of x
int abs(int x)
{
    if (x < 0)
        return -x; // early return (only when x < 0)

    return x;
}

int main()
{
    std::cout << abs(4) << '\n'; // prints 4
    std::cout << abs(-3) << '\n'; // prints 3

    return 0;
}
```

When `abs(4)` is called, `x` has value `4`. `if (x < 0)` is false, so the early return does not execute. The function returns `x` (value `4`) to the caller at the end of the function.

When `abs(-3)` is called, `x` has value `-3`. `if (x < 0)` is true, so the early return executes. The function returns `-x` (value `3`) to the caller at this point.

Historically, early returns were frowned upon. However, in modern programming they are more accepted, particularly when they can be used to make a function simpler, or are used to abort a function early due to some error condition.

> **Related content**
>
> We discuss the debate over early returns further in lesson 8.11 – Break and continue

We'll continue our exploration of if-statements in future lesson 8.2 – If statements and blocks.

Quiz time

Question 1

What is an early return, and what is its behavior?

An early return is a return statement that occurs before the last line of a function. It causes the function to return to the caller immediately.

Question 2

A prime number is a whole number greater than 1 that can only be divided evenly by 1 and itself. Write a program that asks the user to enter a number 0 through 9 (inclusive). If the user enters a number within this range that is prime (2, 3, 5, or 7), print "The digit is prime". Otherwise, print "The digit is not prime".

> **Hint:** Use a chain of *if-else statements* to compare the number the user entered to the prime numbers to see if there is a match.

```cpp
#include <iostream>

bool isPrime(int x)
{
    if (x == 2) // if user entered 2, the digit is prime
        return true;
    else if (x == 3) // if user entered 3, the digit is prime
        return true;
    else if (x == 5) // if user entered 5, the digit is prime
        return true;
    else if (x == 7) // if user entered 7, the digit is prime
        return true;

    return false; // if the user did not enter 2, 3, 5, 7, the digit must not be prime
}

int main()
{
    std::cout << "Enter a number 0 through 9: ";
    int x {};
    std::cin >> x;

    if ( isPrime(x) )
        std::cout << "The digit is prime\n";
    else
        std::cout << "The digit is not prime\n";

    return 0;
}
```

For advanced readers

If the `isPrime()` function above seems a bit verbose/repetitive – it is. We can write `isPrime()` more compactly and efficiently using some concepts that we'll explain in future lessons.

Using the logical OR (||) operator (6.8 – Logical operators):

```cpp
bool isPrime(int x)
{
    return x == 2 || x == 3 || x == 5 || x == 7; // if user entered 2 or 3 or 5 or 7 the digit is prime
}
```

Using a switch statement (8.5 – Switch statement basics):

```
bool isPrime(int x)
{
    switch (x)
    {
        case 2: // if the user entered 2
        case 3: // or if the user entered 3
        case 5: // or if the user entered 5
        case 7: // or if the user entered 7
            return true; // then the digit is prime
    }

    return false; // otherwise the digit must not be prime
}
```

Question 3

How can the length of the following code be reduced (without changing the formatting)?

```
#include <iostream>

bool isAllowedToTakeFunRide()
{
  std::cout << "How tall are you? (cm)\n";

  double height{};
  std::cin >> height;

  if (height >= 140.0)
    return true;
  else
    return false;
}

int main()
{
  if (isAllowedToTakeFunRide())
    std::cout << "Have fun!\n";
  else
    std::cout << "Sorry, you're too short.\n";

  return 0;
}
```

We don't need the if-statement in `isAllowedToTakeFunRide()`. The expression `height >= 140.0` evaluates to a `bool`, which can be directly returned.

```
bool isAllowedToTakeFunRide()
{
  std::cout << "How tall are you? (cm)\n";

  double height{};
  std::cin >> height;

  return height >= 140.0;
}
```

You never need an if-statement of the form:

```
if (condition)
  return true;
else
  return false;
```

This can be replaced by the single statement `return condition`.

4.11 — Chars

To this point, the fundamental data types we've looked at have been used to hold numbers (integers and floating points) or true/false values (Booleans). But what if we want to store letters or punctuation?

```
#include <iostream>

int main()
{
    std::cout << "Would you like a burrito? (y/n)";

    // We want the user to enter a 'y' or 'n' character
    // How do we do this?

    return 0;
}
```

The **char** data type was designed to hold a single `character`. A **character** can be a single letter, number, symbol, or whitespace.

The char data type is an integral type, meaning the underlying value is stored as an integer. Similar to how a Boolean value `0` is interpreted as `false` and non-zero is interpreted as `true`, the integer stored by a `char` variable are intepreted as an `ASCII character`.

ASCII stands for American Standard Code for Information Interchange, and it defines a particular way to represent English characters (plus a few other symbols) as numbers between 0 and 127 (called an **ASCII code** or **code point**). For example, ASCII code 97 is interpreted as the character 'a'.

Character literals are always placed between single quotes (e.g. 'g', '1', ' ').

Here's a full table of ASCII characters:

Code	Symbol	Code	Symbol	Code	Symbol	Code	Symbol
0	NUL (null)	32	(space)	64	@	96	`
1	SOH (start of header)	33	!	65	A	97	a
2	STX (start of text)	34	”	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	’	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	HT (horizontal tab)	41	)	73	I	105	i
10	LF (line feed/new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (form feed / new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (data control 1)	49	1	81	Q	113	q
18	DC2 (data control 2)	50	2	82	R	114	r
19	DC3 (data control 3)	51	3	83	S	115	s
20	DC4 (data control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of transmission block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL (delete)

Codes 0-31 and 127 are called the unprintable chars. These codes were designed to control peripheral devices such as printers

(e.g. by instructing the printer how to move the print head). Most of these are obsolete now. If you try to print these chars, the results are dependent upon your OS (you may get some emoji-like characters).

Codes 32-126 are called the printable characters, and they represent the letters, number characters, and punctuation that most computers use to display basic English text.

If you try to print a character whose value is outside the range of ASCII, the results are also dependent upon your OS.

Initializing chars

You can initialize char variables using character literals:

```
char ch2{ 'a' }; // initialize with code point for 'a' (stored as integer 97) (preferred)
```

You can initialize chars with integers as well, but this should be avoided if possible

```
char ch1{ 97 }; // initialize with integer 97 ('a') (not preferred)
```

> **Warning**
>
> Be careful not to mix up character numbers with integer numbers. The following two initializations are not the same:
>
> ```
> char ch{5}; // initialize with integer 5 (stored as integer 5)
> char ch{'5'}; // initialize with code point for '5' (stored as integer 53)
> ```
>
> Character numbers are intended to be used when we want to represent numbers as text, rather than as numbers to apply mathematical operations to.

Printing chars

When using std::cout to print a char, std::cout outputs the char variable as an ASCII character:

```
#include <iostream>

int main()
{
    char ch1{ 'a' }; // (preferred)
    std::cout << ch1; // cout prints character 'a'

    char ch2{ 98 }; // code point for 'b' (not preferred)
    std::cout << ch2; // cout prints a character ('b')

    return 0;
}
```

This produces the result:

```
ab
```

We can also output char literals directly:

```
std::cout << 'c';
```

This produces the result:

```
c
```

Inputting chars

The following program asks the user to input a character, then prints out the character:

```
#include <iostream>

int main()
{
    std::cout << "Input a keyboard character: ";

    char ch{};
    std::cin >> ch;
    std::cout << "You entered: " << ch << '\n';

    return 0;
}
```

Here's the output from one run:

```
Input a keyboard character: q
You entered: q
```

Note that std::cin will let you enter multiple characters. However, variable *ch* can only hold 1 character. Consequently, only the first input character is extracted into variable *ch*. The rest of the user input is left in the input buffer that std::cin uses, and can be extracted with subsequent calls to std::cin.

You can see this behavior in the following example:

```
#include <iostream>

int main()
{
    std::cout << "Input a keyboard character: "; // assume the user enters "abcd" (without quotes)

    char ch{};
    std::cin >> ch; // ch = 'a', "bcd" is left queued.
    std::cout << "You entered: " << ch << '\n';

    // Note: The following cin doesn't ask the user for input, it grabs queued input!
    std::cin >> ch; // ch = 'b', "cd" is left queued.
    std::cout << "You entered: " << ch << '\n';

    return 0;
}
```

```
Input a keyboard character: abcd
You entered: a
You entered: b
```

If you want to read in more than one char at a time (e.g. to read in a name, word, or sentence), you'll want to use a string instead of a char. A string is a collection of sequential characters (and thus, a string can hold multiple symbols). We discuss this in upcoming lesson (5.7 – Introduction to std::string).

Extracting whitespace characters

Because extracting input ignores leading whitespace, this can lead to unexpected results when trying to extract whitespace characters to a char variable:

```
#include <iostream>

int main()
{
    std::cout << "Input a keyboard character: "; // assume the user enters "a b" (without quotes)

    char ch{};
    std::cin >> ch; // extracts a, leaves " b\n" in stream
    std::cout << "You entered: " << ch << '\n';

    std::cin >> ch; // skips leading whitespace (the space), extracts b, leaves "\n" in stream
    std::cout << "You entered: " << ch << '\n';

    return 0;
}
```

```
Input a keyboard character: a b
You entered: a
You entered: b
```

In the above example, we may have expected to extract the space, but because leading whitespace is skipped, we extracted the `b` character instead.

One simple way to address this is to use the `std::cin.get()` function to perform the extraction instead, as this function does not ignore leading whitespace:

```
#include <iostream>

int main()
{
    std::cout << "Input a keyboard character: "; // assume the user enters "a b" (without quotes)

    char ch{};
    std::cin.get(ch); // extracts a, leaves " b\n" in stream
    std::cout << "You entered: " << ch << '\n';

    std::cin.get(ch); // extracts space, leaves "b\n" in stream
    std::cout << "You entered: " << ch << '\n';

    return 0;
}
```

```
Input a keyboard character: a b
You entered: a
You entered:
```

Char size, range, and default sign

Char is defined by C++ to always be 1 byte in size. By default, a char may be signed or unsigned (though it's usually signed). If you're using chars to hold ASCII characters, you don't need to specify a sign (since both signed and unsigned chars can hold values between 0 and 127).

If you're using a char to hold small integers (something you should not do unless you're explicitly optimizing for space), you should always specify whether it is signed or unsigned. A signed char can hold a number between -128 and 127. An unsigned char can hold a number between 0 and 255.

Escape sequences

There are some sequences of characters in C++ that have special meaning. These characters are called **escape sequences**. An escape sequence starts with a '' (backslash) character, and then a following letter or number.

You've already seen the most common escape sequence: `'\n'`, which can be used to print a newline:

```
#include <iostream>

int main()
{
    int x { 5 };
    std::cout << "The value of x is: " << x << '\n'; // standalone \n goes in single quotes
    std::cout << "First line\nSecond line\n";        // \n can be embedded in double quotes
    return 0;
}
```

This outputs:

```
The value of x is: 5
First line
Second line
```

Another commonly used escape sequence is `'\t'`, which embeds a horizontal tab:

```
#include <iostream>

int main()
{
    std::cout << "First part\tSecond part";
    return 0;
}
```

Which outputs:

```
First part  Second part
```

Three other notable escape sequences are: `\'` prints a single quote `\"` prints a double quote `\\` prints a backslash

Here's a table of all of the escape sequences:

Name	Symbol	Meaning
Alert		Makes an alert, such as a beep
Backspace	Moves the cursor back one space	
Formfeed	Moves the cursor to next logical page	
Newline		Moves cursor to next line
Carriage return	Moves cursor to beginning of line	
Horizontal tab	Prints a horizontal tab	
Vertical tab	Prints a vertical tab	
Single quote	'	Prints a single quote
Double quote	"	Prints a double quote
Backslash	\	Prints a backslash.
Question mark	?	Prints a question mark. No longer relevant. You can use question marks unescaped.
Octal number	(number)	Translates into char represented by octal
Hex number	(number)	Translates into char represented by hex number

Here are some examples:

```
#include <iostream>

int main()
{
    std::cout << "\"This is quoted text\"\n";
    std::cout << "This string contains a single backslash \\\n";
    std::cout << "6F in hex is char '\x6F'\n";
    return 0;
}
```

Prints:

```
"This is quoted text"
This string contains a single backslash \
6F in hex is char 'o'
```

> **Warning**
>
> Escape sequences start with a backslash (\), not a forward slash (/). If you use a forward slash by accident, it may still compile, but will not yield the desired result.

Newline () vs. std::endl

We cover this topic in lesson 1.5 – Introduction to iostream: cout, cin, and endl.

What's the difference between putting symbols in single and double quotes?

Text between single quotes is treated as a `char` literal, which represents a single character. For example, `'a'` represents the character `a`, `'+'` represents the character for the plus symbol, `'5'` represents the character `5` (not the number 5), and `'\n'` represents the newline character.

Text between double quotes (e.g. "Hello, world!") is treated as a C-style string literal, which can contain multiple characters. We discuss strings in lesson 5.2 – Literals.

> **Best practice**
>
> Single characters should usually be single-quoted, not double-quoted (e.g. `'t'` or `'\n'`, not `"t"` or `"\n"`). One possible exception occurs when doing output, where it can be preferential to double quote everything for consistency (see lesson 1.5 – Introduction to iostream: cout, cin, and endl).

Avoid multicharacter literals

For backwards compatibility reasons, many C++ compilers support **multicharacter literals**, which are char literals that contain multiple characters (e.g. `'56'`). If supported, these have an implementation-defined value (meaning it varies depending on the compiler). Because they are not part of the C++ standard, and their value is not strictly defined, multicharacter literals should be avoided.

> **Best practice**
>
> Avoid multicharacter literals (e.g.

4.12 — Introduction to type conversion and static_cast

Implicit type conversion

Consider the following program:

```
#include <iostream>

void print(double x) // print takes a double parameter
{
    std::cout << x << '\n';
}

int main()
{
    print(5); // what happens when we pass an int value?

    return 0;
}
```

In the above example, the `print()` function has a parameter of type `double` but the caller is passing in the value `5` which is of type `int`. What happens in this case?

In most cases, C++ will allow us to convert values of one fundamental type to another fundamental type. The process of converting data from one type to another type is called **type conversion**. Thus, the int argument `5` will be converted to double value `5.0` and then copied into parameter `x`. The `print()` function will print this value, resulting in the following output:

```
5
```

> **A reminder**
>
> By default, floating point values whose decimal part is 0 print without the decimal places (e.g. `5.0` prints as `5`).

When the compiler does type conversion on our behalf without us explicitly asking, we call this **implicit type conversion**. The above example illustrates this – nowhere do we explicitly tell the compiler to convert integer value `5` to double value `5.0`. Rather, the function is expecting a double value, and we pass in an integer argument. The compiler will notice the mismatch and implicitly convert the integer to a double.

Here's a similar example where our argument is an int variable instead of an int literal:

```cpp
#include <iostream>

void print(double x) // print takes a double parameter
{
    std::cout << x << '\n';
}

int main()
{
    int y { 5 };
    print(y); // y is of type int

    return 0;
}
```

This works identically to the above. The value held by int variable `y` (`5`) will be converted to double value `5.0`, and then copied into parameter `x`.

Type conversion of a value produces a new value

The type conversion process does not modify the value (or object) supplying the data to be converted. Instead, the conversion process uses that data as input, and produces the converted result.

> **Key insight**
>
> The type conversion of a value to another type of value behaves much like a call to a function whose return type matches the target type of the conversion. The data to be converted is passed in as an argument, and the converted result is returned (in a temporary object) to be used by the caller.

In the above example, the conversion does not change variable `y` from type `int` to `double` or the value of `y` from `5` to `5.0`. Instead, the conversion uses the value of `y` (`5`) as input, and returns a temporary object of type `double` with value `5.0`. This temporary object is then passed to function `print`.

> **For advanced readers**
>
> Some advanced type conversions (e.g. those involving `const_cast` or `reinterpret_cast`) do not return temporary objects, but instead reinterpret the type of an existing value or object.

Implicit type conversion warnings

Although implicit type conversion is sufficient for most cases where type conversion is needed, there are a few cases where it is not. Consider the following program, which is similar to the example above:

```
#include <iostream>

void print(int x) // print now takes an int parameter
{
    std::cout << x << '\n';
}

int main()
{
    print(5.5); // warning: we're passing in a double value

    return 0;
}
```

In this program, we've changed `print()` to take an `int` parameter, and the function call to `print()` is now passing in `double` value `5.5`. Similar to the above, the compiler will use implicit type conversion in order to convert double value `5.5` into a value of type `int`, so that it can be passed to function `print()`.

Unlike the initial example, when this program is compiled, your compiler will generate some kind of a warning about a possible loss of data. And because you have "treat warnings as errors" turned on (you do, right?), your compiler will abort the compilation process.

> **Tip**
>
> You'll need to disable "treat warnings as errors" temporarily if you want to compile this example. See lesson 0.11 – Configuring your compiler: Warning and error levels for more information about this setting.

When compiled and run, this program prints the following:

```
5
```

Note that although we passed in value `5.5`, the program printed `5`. Because integral values can't hold fractions, when double value `5.5` is implicitly converted to an `int`, the fractional component is dropped, and only the integral value is retained.

Because converting a floating point value to an integral value results in any fractional component being dropped, the compiler will warn us when it does an implicit type conversion from a floating point to an integral value. This happens even if we were to pass in a floating point value with no fractional component, like `5.0` – no actual loss of value occurs during the conversion to integral value `5` in this specific case, but the compiler may still warn us that the conversion is unsafe.

> **Key insight**
>
> Some type conversions (such as a `char` to an `int`) always preserve the value being converted, whereas others (such as `double` to `int`) may result in the value being changed during conversion. Unsafe implicit conversions will typically either generate a compiler warning, or (in the case of brace initialization) an error.

This is one of the primary reasons brace initialization is the preferred initialization form. Brace initialization will ensure we don't try to initialize a variable with an initializer that will lose value when it is implicitly type converted:

```
int main()
{
    double d { 5 }; // okay: int to double is safe
    int x { 5.5 }; // error: double to int not safe

    return 0;
}
```

Related content

Implicit type conversion is a meaty topic. We dig into this topic in more depth in future lessons, starting with lesson 10.1 – Implicit type conversion.

An introduction to explicit type conversion via the static_cast operator

Back to our most recent `print()` example, what if we *intentionally* wanted to pass a double value to a function taking an integer (knowing that the converted value would drop any fractional component?) Turning off "treat warnings as errors" just to make our program compile is a bad idea, because then we'll have warnings every time we compile (which we will quickly learn to ignore), and we risk overlooking warnings about more serious issues.

C++ supports a second method of type conversion, called explicit type conversion. **Explicit type conversion** allow us (the programmer) to explicitly tell the compiler to convert a value from one type to another type, and that we take full responsibility for the result of that conversion. If such a conversion results in the loss of value, the compiler will not warn us.

To perform an explicit type conversion, in most cases we'll use the `static_cast` operator. The syntax for the `static cast` looks a little funny:

```
static_cast<new_type>(expression)
```

static_cast takes the value from an expression as input, and returns that value converted into the type specified by *new_type* (e.g. int, bool, char, double).

Key insight

Whenever you see C++ syntax (excluding the preprocessor) that makes use of angled brackets (`<>`), the thing between the angled brackets will most likely be a type. This is typically how C++ deals with code that need a parameterized type.

Let's update our prior program using `static_cast` :

```
#include <iostream>

void print(int x)
{
    std::cout << x << '\n';
}

int main()
{
    print( static_cast<int>(5.5) ); // explicitly convert double value 5.5 to an int

    return 0;
}
```

Because we're now explicitly requesting that double value `5.5` be converted to an `int` value, the compiler will not generate a warning about a possible loss of data upon compilation (meaning we can leave "treat warnings as errors" enabled).

> **Related content**
>
> C++ supports other types of casts. We talk more about the different types of casts in future lesson 10.6 – Explicit type conversion (casting) and static_cast.

Using static_cast to convert char to int

In the lesson on chars 4.11 – Chars, we saw that printing a char value using `std::cout` results in the value being printed as a char:

```
#include <iostream>

int main()
{
    char ch{ 97 }; // 97 is ASCII code for 'a'
    std::cout << ch << '\n';

    return 0;
}
```

This prints:

```
a
```

If we want to print the integral value instead of the char, we can do this by using `static_cast` to cast the value from a `char` to an `int`:

```
#include <iostream>

int main()
{
    char ch{ 97 }; // 97 is ASCII code for 'a'
    // print value of variable ch as an int
    std::cout << ch << " has value " << static_cast<int>(ch) << '\n';

    return 0;
}
```

This prints:

```
a has value 97
```

It's worth noting that the argument to *static_cast* evaluates as an expression. When we pass in a variable, that variable is evaluated to produce its value, and that value is then converted to the new type. The variable itself is *not* affected by casting its value to a new type. In the above case, variable `ch` is still a char, and still holds the same value even after we've cast its value to an `int`.

Sign conversions using static_cast

Signed integral values can be converted to unsigned integral values, and vice-versa, using a static cast.

If the value being converted can be represented in the destination type, the converted value will remain unchanged (only the type will change). For example:

```
#include <iostream>

int main()
{
    unsigned int u1 { 5 };
    // Convert value of u1 to a signed int
    int s1 { static_cast<int>(u1) };
    std::cout << s1 << '\n'; // prints 5

    int s2 { 5 };
    // Convert value of s2 to an unsigned int
    unsigned int u2 { static_cast<unsigned int>(s2) };
    std::cout << u2 << '\n'; // prints 5

    return 0;
}
```

This prints:

```
5
5
```

Since the value `5` is in the range of both a signed int and an unsigned int, the value `5` can be converted to either type without issue.

If the value being converted cannot be represented in the destination type:

- If the destination type is unsigned, the value will be modulo wrapped. We cover modulo wrapping in lesson 4.5 – Unsigned integers, and why to avoid them.
- If the destination type is signed, the value is implementation-defined prior to C++20, and will be modulo wrapped as of C++20.

Here's an example of converting two values that are not representable in the destination type (assuming 32-bit integers):

```
#include <iostream>

int main()
{
    int s { -1 };
    std::cout << static_cast<unsigned int>(s) << '\n'; // prints 4294967295

    unsigned int u { 4294967295 }; // largest 32-bit unsigned int
    std::cout << static_cast<int>(u) << '\n'; // implementation-defined prior to C++20, -1 as of C++20

    return 0;
}
```

As of C++20, this produces the result:

```
4294967295
-1
```

Signed int value `-1` cannot be represented as an unsigned int. The result modulo wraps to unsigned int value `4294967295`.

Unsigned int value `4294967295` cannot be represented as a signed int. Prior to C++20, the result is implementation defined (but will probably be `-1`). As of C++20, the result will modulo wrap to `-1`.

> **Warning**
>
> Converting an unsigned integral value to a signed integral value will result in implementation-defined behavior prior to C++20 if the value being converted can not be represented in the signed type.

std::int8_t and std::uint8_t likely behave like chars instead of integers

As noted in lesson 4.6 – Fixed-width integers and size_t, most compilers define and treat `std::int8_t` and `std::uint8_t` (and the corresponding fast and least fixed-width types) identically to types `signed char` and `unsigned char` respectively. Now that we've covered what chars are, we can demonstrate where this can be problematic:

```
#include <cstdint>
#include <iostream>

int main()
{
    std::int8_t myInt{65};      // initialize myInt with value 65
    std::cout << myInt << '\n'; // you're probably expecting this to print 65

    return 0;
}
```

Because `std::int8_t` describes itself as an int, you might be tricked into believing that the above program will print the integral

value `65`. However, on most systems, this program will print `A` instead (treating `myInt` as a `signed char`). However, this is not guaranteed (on some systems, it may actually print `65`).

If you want to ensure that a `std::int8_t` or `std::uint8_t` object is treated as an integer, you can convert the value to an integer using `static_cast`:

```
#include <cstdint>
#include <iostream>

int main()
{
    std::int8_t myInt{65};
    std::cout << static_cast<int>(myInt) << '\n'; // will always print 65

    return 0;
}
```

In cases where `std::int8_t` is treated as a char, input from the console can also cause problems:

```
#include <cstdint>
#include <iostream>

int main()
{
    std::cout << "Enter a number between 0 and 127: ";
    std::int8_t myInt{};
    std::cin >> myInt;

    std::cout << "You entered: " << static_cast<int>(myInt) << '\n';

    return 0;
}
```

A sample run of this program:

```
Enter a number between 0 and 127: 35
You entered: 51
```

Here's what's happening. When `std::int8_t` is treated as a char, the input routines interpret our input as a sequence of characters, not as an integer. So when we enter `35`, we're actually entering two chars, `'3'` and `'5'`. Because a char object can only hold one character, the `'3'` is extracted (the `'5'` is left in the input stream for possible extraction later). Because the char `'3'` has ASCII code point 51, the value `51` is stored in `myInt`, which we then print later as an int.

In contrast, the other fixed-width types will always print and input as integral values.

Quiz time

Question 1

Write a short program where the user is asked to enter a single character. Print the value of the character and its ASCII code, using `static_cast`.

The program's output should match the following:

```
Enter a single character: a
You entered 'a', which has ASCII code 97.
```

Show Solution

```cpp
#include <iostream>

int main()
{
  std::cout << "Enter a single character: ";
  char c{};
  std::cin >> c;
  std::cout << "You entered '" << c << "', which has ASCII code " << static_cast<int>(c) << ".\n";

  return 0;
}
```

Question 2

Modify the program you wrote for quiz #1 to use implicit type conversion instead of `static_cast`. How many different ways can you think of to do this?

Note: You should favor explicit conversions over implicit conversions, so don't actually do this in real programs – this is just to test your understanding of where implicit conversions can occur.

Show Solution

There are a few easy ways to do this.

First, we can create an `int` variable, and initialize it with our `char` value. This will do the implicit conversion on initialization.

```cpp #include

int main() { std::cout << "Enter a single character:"; char c{}; std
```

4.x — Chapter 4 summary and quiz

Chapter Review

The smallest unit of memory is a **binary digit**, also called a **bit**. The smallest unit amount of memory that can be addressed (accessed) directly is a **byte**. The modern standard is that a byte equals 8 bits.

A **data type** tells the compiler how to interpret the contents of memory in some meaningful way.

C++ comes with support for many fundamental data types, including floating point numbers, integers, boolean, chars, null pointers, and void.

Void is used to indicate no type. It is primarily used to indicate that a function does not return a value.

Different types take different amounts of memory, and the amount of memory used may vary by machine.

> **Related content** See 4.3 – Object sizes and the sizeof operator for a table indicating the minimum size for each fundamental type.

The **sizeof** operator can be used to return the size of a type in bytes.

Signed integers are used for holding positive and negative whole numbers, including 0. The set of values that a specific data type can hold is called its **range**. When using integers, keep an eye out for overflow and integer division problems.

Unsigned integers only hold positive numbers (and 0), and should generally be avoided unless you're doing bit-level manipulation.

Fixed-width integers are integers with guaranteed sizes, but they may not exist on all architectures. The fast and least integers are the fastest and smallest integers that are at least some size. `std::int8_t` and `std::uint8_t` should generally be avoided, as they tend to behave like chars instead of integers.

size_t is an unsigned integral type that is used to represent the size or length of objects.

Scientific notation is a shorthand way of writing lengthy numbers. C++ supports scientific notation in conjunction with floating point numbers. The digits in the significand (the part before the e) are called the **significant digits**.

Floating point is a set of types designed to hold real numbers (including those with a fractional component). The **precision** of a number defines how many significant digits it can represent without information loss. A **rounding error** can occur when too many significant digits are stored in a floating point number that can't hold that much precision. Rounding errors happen all the time, even with simple numbers such as 0.1. Because of this, you shouldn't compare floating point numbers directly.

The **Boolean** type is used to store a `true` or `false` value.

If statements allow us to execute one or more lines of code if some condition is true. The conditional expression of an if-statement is interpreted as a boolean value. An **else statement** can be used to execute a statement when a prior if-statement condition evaluates to false.

Char is used to store values that are interpreted as an ASCII character. When using chars, be careful not to mix up ASCII code values and numbers. Printing a char as an integer value requires use of `static_cast`.

Angled brackets are typically used in C++ to represent something that needs a parameterizable type. This is used with `static_cast` to determine what data type the argument should be converted to (e.g. `static_cast<int>(x)` will return the value of `x` as an `int`).

Quiz time

Pick the appropriate data type for a variable in each of the following situations. Be as specific as possible. If the answer is an integer, pick int (if size isn't important), or a specific fixed-width integer type (e.g. std::int16_t) based on range.

a) The age of the user (in years) (assume the size of the type isn't important)

> **Solution** int

b) Whether the user wants the application to check for updates

> **Solution** bool

c) pi (3.14159265)

> **Solution** double

d) The number of pages in a textbook (assume size is not important)

> **Solution** Since books probably won't have more than 32,767 pages, int should be fine here.

e) The length of a couch in feet, to 2 decimal places (assume size is important)

> **Solution** float

f) How many times you've blinked since you were born (note: answer is in the millions)

> **Solution** std::int32_t

g) A user selecting an option from a menu by letter

> **Solution** char

h) The year someone was born (assuming size is important)

> **Solution** std::int16_t. You can use positive numbers to represent AD birthdates, and negative numbers to represent BC birthdates.

> **Author's note**
>
> The quizzes get more challenging starting here. These quizzes that ask you to write a program are designed to ensure you can integrate multiple concepts that have been presented throughout the lessons. You should be prepared to spend some time with these problems. If you're new to programming, you shouldn't expect to be able to answer these immediately.
>
> Remember, the goal here is to help you pinpoint what you know, and which concepts you may need to spend additional time on. If you find yourself struggling a bit, that's okay.
>
> Here are some tips: - Don't try to write the whole solution at once. Write one function, then test it to make sure it works as expected. Then proceed. - Use your debugger to help figure out where things are going wrong. - Go back and review the answers to quizzes from prior lessons in the chapter, as they'll often contain similar concepts.
>
> If you are truly stuck, feel free to look at the solution, but take the time to make sure you understand what each line does before proceeding. As long as you leave understanding the concepts, it doesn't matter so much whether you were able to get it yourself, or had to look at the solution before proceeding.

Write the following program: The user is asked to enter 2 floating point numbers (use doubles). The user is then asked to enter one of the following mathematical symbols: +, -, *, or /. The program computes the answer on the two numbers the user entered and prints the results. If the user enters an invalid symbol, the program should print nothing.

Example of program:

```
Enter a double value: 6.2
Enter a double value: 5
Enter +, -, *, or /: *
6.2 * 5 is 31
```

Hint Hint: Write three functions: one to get a double value, one to get the arithmetic symbol, and one to calculate and print the answer.

Hint Hint: Use if-statements and `operator==` to compare the user input to the desired arithmetic symbol.

Hint Hint: Consider using an early return (covered in lesson 4.10 – Introduction to if statements) if the user doesn't pass in a supported operation.

Solution

```cpp
#include <iostream>

double getDouble()
{
    std::cout << "Enter a double value: ";
    double x{};
    std::cin >> x;
    return x;
}

char getOperator()
{
    std::cout << "Enter +, -, *, or /: ";
    char operation{};
    std::cin >> operation;
    return operation;
}

void printResult(double x, char operation, double y)
{
    double result{};

    if (operation == '+')
        result = x + y;
    else if (operation == '-')
        result = x - y;
    else if (operation == '*')
        result = x * y;
    else if (operation == '/')
        result = x / y;
    else        // if the user did not pass in a supported operation
        return; // early return

    std::cout << x << ' ' << operation << ' ' << y << " is " << result << '\n';
}

int main()
{
    double x { getDouble() };
    double y { getDouble() };

    char operation { getOperator() };

    printResult(x, operation, y);

    return 0;
}
```

Extra credit: This one is a little more challenging.

Write a short program to simulate a ball being dropped off of a tower. To start, the user should be asked for the height of the tower

in meters. Assume normal gravity (9.8 m/s2), and that the ball has no initial velocity (the ball is not moving to start). Have the program output the height of the ball above the ground after 0, 1, 2, 3, 4, and 5 seconds. The ball should not go underneath the ground (height 0).

Use a function to calculate the height of the ball after x seconds. The function can calculate how far the ball has fallen after x seconds using the following formula: distance fallen = gravity_constant * x_seconds2 / 2

Expected output:

```
Enter the height of the tower in meters: 100
At 0 seconds, the ball is at height: 100 meters
At 1 seconds, the ball is at height: 95.1 meters
At 2 seconds, the ball is at height: 80.4 meters
At 3 seconds, the ball is at height: 55.9 meters
At 4 seconds, the ball is at height: 21.6 meters
At 5 seconds, the ball is on the ground.
```

Note: Depending on the height of the tower, the ball may not reach the ground in 5 seconds – that's okay. We'll improve this program once we've covered loops.

Note: The `^` symbol isn't an exponent in C++. Implement the formula using multiplication instead of exponentiation.

Note: Remember to use double literals for doubles, e.g. `2.0` rather than `2`.

Solution

```cpp
#include <iostream>

// Gets tower height from user and returns it
double getTowerHeight()
{
  std::cout << "Enter the height of the tower in meters: ";
  double towerHeight{};
  std::cin >> towerHeight;
  return towerHeight;
}

// Returns the current ball height after "seconds" seconds
double calculateBallHeight(double towerHeight, int seconds)
{
  double gravity { 9.8 };

  // Using formula: s = (u * t) + (a * t^2) / 2
  // here u (initial velocity) = 0, so (u * t) = 0
  double fallDistance { gravity * (seconds * seconds) / 2.0 };
  double ballHeight { towerHeight - fallDistance };

  // If the ball would be under the ground, place it on the ground
  if (ballHeight < 0.0)
```

```
        return 0.0;

    return ballHeight;
}

// Prints ball height above ground
void printBallHeight(double ballHeight, int seconds)
{
    if (ballHeight > 0.0)
        std::cout << "At " << seconds << " seconds, the ball is at height: " << ballHeight << " meters\n";
    else
        std::cout << "At " << seconds << " seconds, the ball is on the ground.\n";
}

// Calculates the current ball height and then prints it
// This is a helper function to make it easier to do this
void calculateAndPrintBallHeight(double towerHeight, int seconds)
{
    double ballHeight{ calculateBallHeight(towerHeight, seconds) };
    printBallHeight(ballHeight, seconds);
}

int main()
{
    double towerHeight{ getTowerHeight() };

    calculateAndPrintBallHeight(towerHeight, 0);
    calculateAndPrintBallHeight(towerHeight, 1);
    calculateAndPrintBallHeight(towerHeight, 2);
    calculateAndPrintBallHeight(towerHeight, 3);
    calculateAndPrintBallHeight(towerHeight, 4);
    calculateAndPrintBallHeight(towerHeight, 5);

    return 0;
}
```

For this program, we have 3 tasks: get the initial tower height, calculate the ball's current height, and print the ball's current height. Under the best practice that function should do one thing, we have a different function for each of these tasks. `calculateAndPrintBallHeight()` is a helper function that makes calculating and printing the height more readable when called from `main()`.